

DATA MODELING

FOR ANALYTICS ENGINEERING

MAVEN® ANALYTICS

*Copyright Maven Analytics, LLC

MEET YOUR INSTRUCTOR



ALICE ZHAO

Alice is the author of the book, SQL Pocket Guide, and an adjunct lecturer for Northwestern University's Machine Learning and Data Science master's program.

She is a Staff Instructor at Maven Analytics, with a focus on data science & analytics engineering.

- ✓ **Lead Data Science Instructor**
- ✓ **Author & University Lecturer**

*Copyright Maven Analytics, LLC

COURSE OUTLINE

DATA MODELING THEORY

1	**Data Modeling Overview**	*Understand what a data model is and the difference between modeling for OLTP vs OLAP databases (aka dimensional modeling)*
2	**Table Fundamentals**	*Learn about table structure, including rows, columns, data types, constraints, primary keys, foreign keys, and grain / level of detail*
3	**Fact & Dimension Tables**	*Get introduced to the core concept of dimensional modeling – transforming transactional databases into fact & dimension tables*
4	**Schema Designs**	*See examples of popular schema designs including the star schema, snowflake schema and a transactional schema*
5	**Handling Data Changes**	*Learn about different ways to handle data changes in a database, with a focus on slowly changing dimensions (SCD)*

PRACTICE

6	**Modern Data Workflow**	*Walk through the steps of the modern data workflow: scoping, ingesting, transforming, orchestrating, consuming, and monitoring*
7	**Data Modeling Process**	*Go through the dimensional modeling process: specifying fact & dimension tables, deciding on columns & SCD types, and more*

*Copyright Maven Analytics, LLC

SETTING EXPECTATIONS



Our **Analytics Engineering 101** course is a prerequisite for this course

- *We will do a quick recap of the concepts as part of this course, but you are expected to understand what an Analytics Engineer does, why they're needed, and what it means to have analytics-ready data*



We'll be taking a conceptual approach here – **this is not a coding course!**

- *You'll learn how to design OLAP models for data warehouses from a practical lens, using a clear framework*



We'll **practice applying the theory** with a final data modeling project

- *At the end of this course, we'll walk step-by-step through the logic of how to transform data from a raw transactional database to an analytics-ready dimensional model*
- *We'll be implementing this design in SQL using dbt and Snowflake later in this learning path*

*Copyright Maven Analytics, LLC

RECAP: ANALYTICS ENGINEERING 101 (OVERVIEW)



Analytics engineering sits between data engineering and data analysis

- *The job of an analytics engineer is to take raw data and transform it into analytics-ready data for data analysts, data scientists and BI developers to use for analysis and dashboards*



The **modern data stack** refers to a collection of tools used in the cloud era

- *These include data ingestion tools like Fivetran, data warehouses like Snowflake, BigQuery and Redshift, transformation tools like dbt, orchestration tools like Airflow and BI tools like Tableau and PowerBI*



Analytics engineers **centralize & organize** business logic in data warehouses

- *By doing so, different teams can have a single source of truth to build dashboards, tables that are well-structured to easily query, and a place to track historical changes – all making life easier for data analysis*



There is a lot to learn, but the main goal is to **structure your thinking**

- *We want you to understand core data concepts, and how various tools fit and flow together. Don't be intimidated by logo charts! Tools will continue to evolve over time, but the foundational ideas stay the same*

*Copyright Maven Analytics, LLC

RECAP: ANALYTICS ENGINEERING 101 (DATA TERMS)

- **Databases** store and organize data, and **data warehouses** are specialized for storing analytics-ready data for various teams
  - *Other storage options include data lakes (unorganized storage) & lakehouses (mix of data lake & warehouse)*

- **OLTP** is for transactional systems, **OLAP** is for analytical systems
  - *OLTP databases store transactional data to record events and are optimized for writes & updates*
  - *OLAP databases store aggregated data for reporting & analytics and are optimized for querying*

- **ETL** and **ELT** are approaches for loading data into a data warehouse
  - *ETL is an older technique where the transformations happen before the load due to expensive on-prem DWs*
  - *ELT is a newer approach where the data is first loaded into a cloud data warehouse and transformed there*

- The job of an AE is to turn **OLTP** data into **OLAP** data using **ELT**

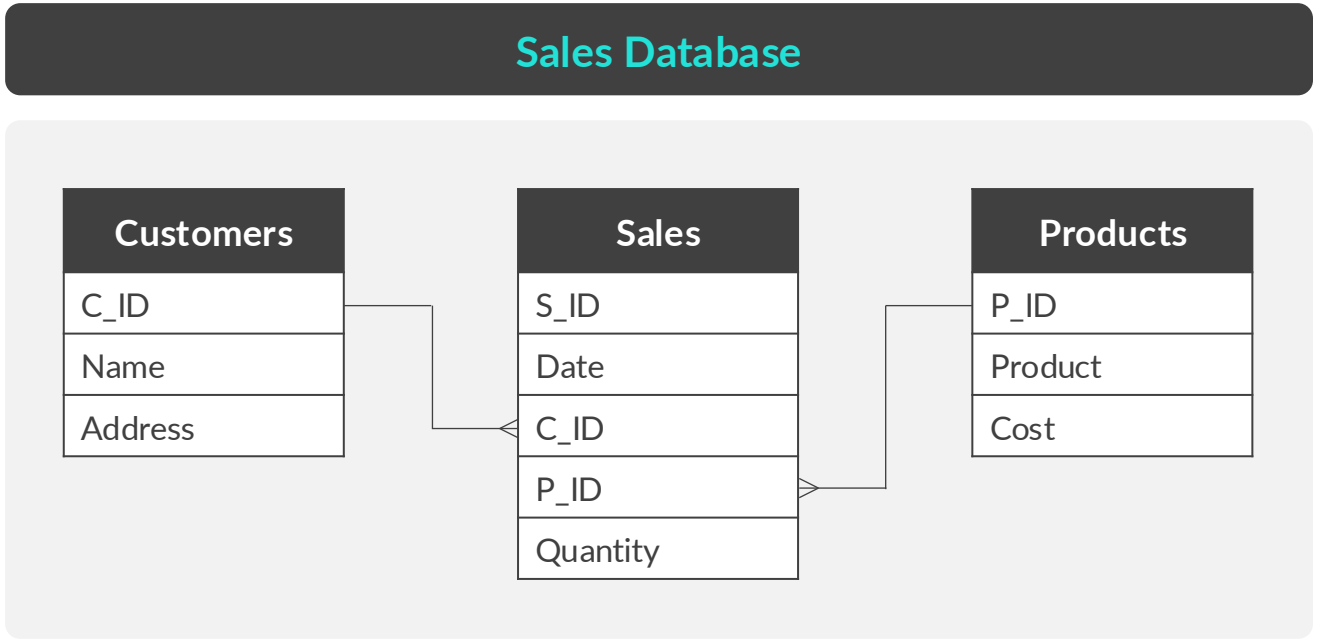
*Copyright Maven Analytics, LLC

DATA MODELING OVERVIEW

*Copyright Maven Analytics, LLC

WHAT IS A DATA MODEL?

A **data model** is a blueprint that shows how your data is organized and connected



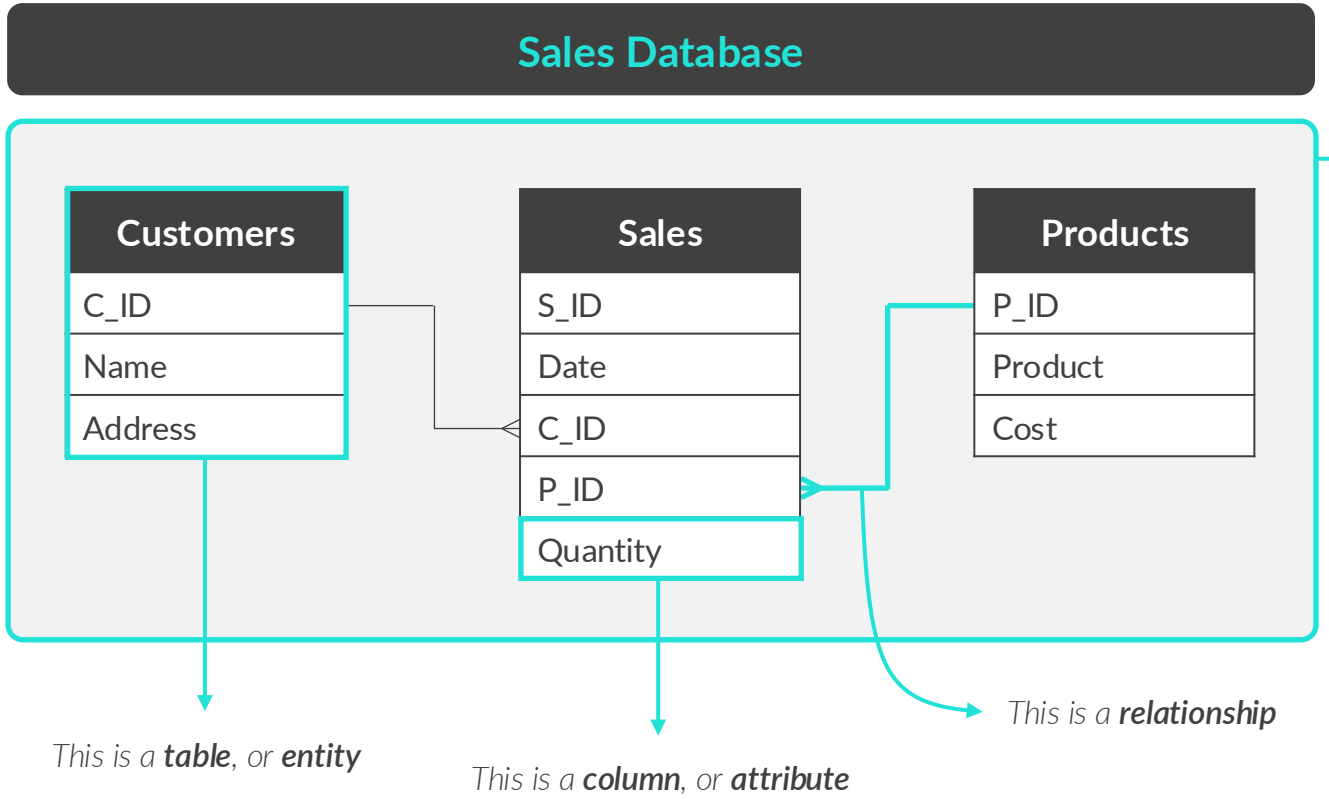
*The data in this **Sales Database** is organized into three **tables** (Customers, Sales, and Products) and is connected by related **columns** (C_ID and P_ID)*

Data models typically only contain structural information like the table and column names shown here, but we're also going to **include rows of sample data**, for illustration purposes, throughout this course

*Copyright Maven Analytics, LLC

DATA MODELING TERMS

Common conceptual **data modeling terms** include entities, relationships, attributes, and schemas



In some tools, **database** and **schema** are used interchangeably; in others, like Snowflake, a database will contain multiple schemas – you'll encounter both usages in the wild

*Copyright Maven Analytics, LLC

OLTP DATA MODELS

OLTP data models are optimized to **record and store** events (or transactions) as they happen

Sales Database: OLTP Data Model

Customers

C_ID	Name	Address
C001	Ava	12 Main Street
C002	Ben	45 Oak Avenue
C003	Cal	7 Maple Road
C004	Deb	301 Pine Street
C005	Eve	88 Elm Drive

Products

P_ID	Product	Cost
P16	Pretzels	$3.99
P17	Chocolate	$4.50
P18	Chips	$2.99
P19	Soda	$1.50

Sales

S_ID	Date	C_ID	P_ID	Quantity
101	5/1/26	C001	P19	1
102	5/1/26	C002	P17	1
103	5/1/26	C002	P18	1
104	5/1/26	C003	P16	5
105	5/2/26	C001	P19	1
106	5/2/26	C004	P17	3
107	5/2/26	C005	P18	2

*Each sales row represents a **single event***

*All tables are **optimized for inserts** (adding rows)*

*Copyright Maven Analytics, LLC

OLAP DATA MODELS

OLAP data models are OLTP models transformed by analytics engineers for **reporting and analytics**

Sales Database: OLAP Data Model

dim_customer

C_Key	C_ID	Name	Address
CK1	C001	Ava	12 Main Street
CK2	C002	Ben	45 Oak Avenue
CK3	C003	Cal	7 Maple Road

*New **surrogate keys** were added to each table (more on this later!)*

dim_product

P_Key	P_ID	Product	Cost
PK16	P16	Pretzels	$3.99
PK17	P17	Chocolate	$4.50
PK18	P18	Chips	$2.99

dim_date

D_Key	Date	Is_Weekend
DK260501	5/1/26	0
DK260502	5/2/26	1

*This **lookup table** was added with details for each date*

fct_sales

S_Key	S_ID	C_Key	P_Key	D_Key	Quantity	Revenue
SK1	101	CK1	PK19	DK260501	1	$1.50
SK2	102	CK2	PK17	DK260501	1	$4.50
SK3	103	CK2	PK18	DK260501	1	$2.99
SK4	104	CK3	PK16	DK260501	5	$19.95
SK5	105	CK1	PK19	DK260502	1	$1.50

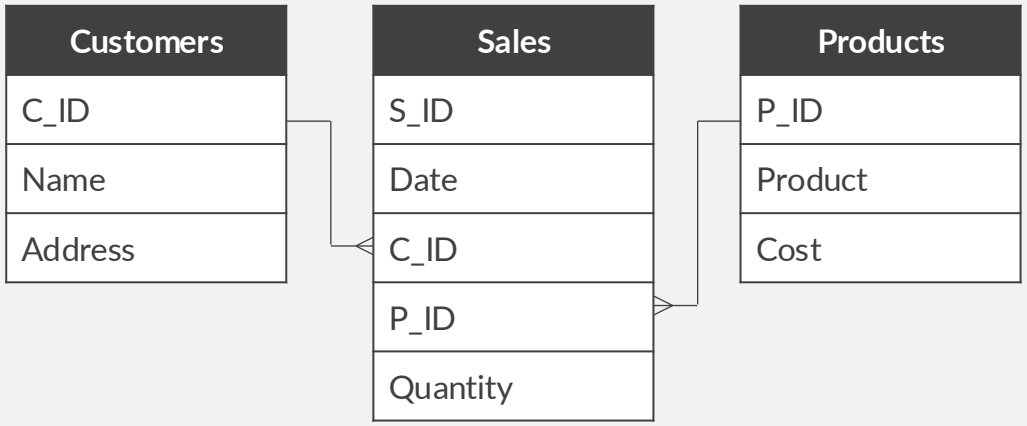
*This central sales table now contains an **engineered column** for analytics*

*Copyright Maven Analytics, LLC

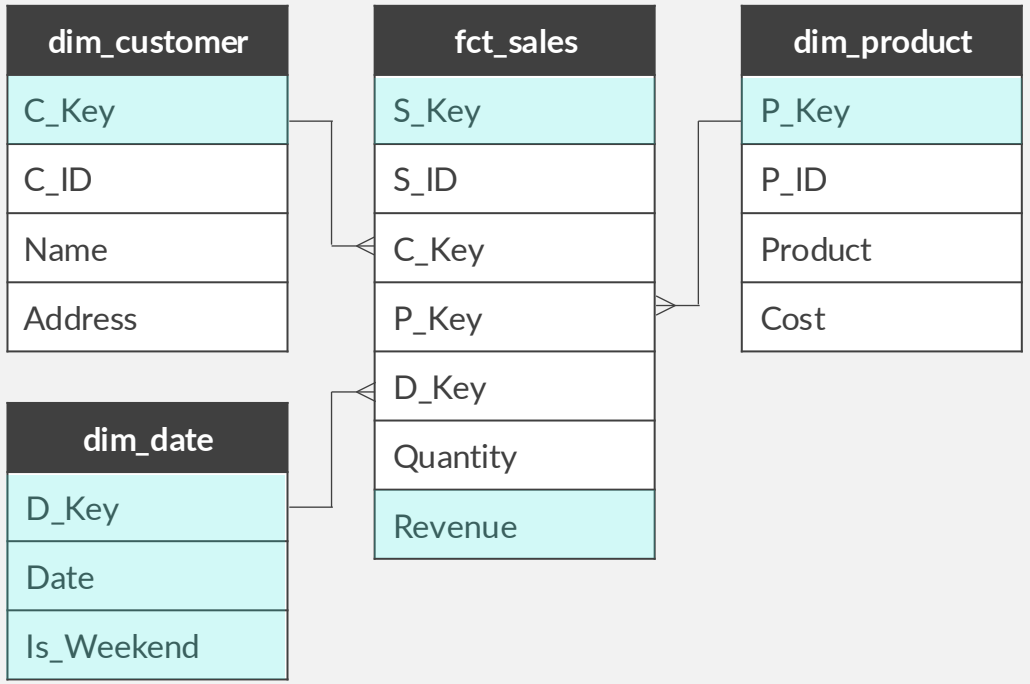
DIMENSIONAL MODELING

This transformation process of turning an OLTP model into an OLAP model by analytics engineers is called data modeling or, more specifically, **dimensional modeling**

OLTP Data Model



OLAP Data Model



*Copyright Maven Analytics, LLC

TABLE FUNDAMENTALS

*Copyright Maven Analytics, LLC

TABLE FUNDAMENTALS

A **table**, the main component in a data model, is made up of rows and columns

- Each **column** holds a single data type *(integer, date, text)*, and can have additional constraints *(unique values, nulls)*
- Each **row**, or record, stores the attributes for a single observation

Column

ID	Date	Customer	Product	Quantity
101	5/1/26	C001	P19	1
102	5/1/26	C002	P17	1
103	5/1/26	C002	P18	NULL
104	5/1/26	C003	P16	5
105	5/2/26	C001	P19	1
106	5/2/26	C004	P17	3
107	5/2/26	C005	P18	2

Row



Many of the concepts in this section apply to both OLTP and OLAP databases, but **we will be approaching everything with an OLAP mindset** in this course

*Copyright Maven Analytics, LLC

GRAIN

The **grain**, or level of detail, defines what a single row in a table represents

- When creating a new table for dimensional modeling, the first step is to decide on the grain

Grain: Sale for a particular customer / product / date

ID	Date	Customer	Product	Quantity
101	5/1/26	C001	P19	1
102	5/1/26	C002	P17	1
103	5/1/26	C002	P18	3

Customer C002 bought one unit of product P17 on 5/1/26

Grain: Daily sales by region & product

Date	Region	Product	Units	Revenue
5/1/26	North	P17	29	$43.50
5/1/26	South	P17	43	$64.50
5/2/26	North	P17	52	$78.00

The South region sold 43 units of product P17 for $64.50 in revenue on 5/1/26

Both of these are valid grain options – the "correct" grain depends on what questions you're trying to answer:

"How much of each product has customer C002 bought this year?"

You need the first table

"Which region makes the most revenue from product P17?"

The second table is more suitable

*Copyright Maven Analytics, LLC

GRAIN FAQs



Why would you aggregate the data? Don't you want to keep all the details?

- Aggregating data reduces table size and can improve query performance



How do I know which grain to pick?

- Start with the questions you're frequently trying to answer
- Keep the fine-grain if details are needed, but aggregate them if reporting is mostly based on summary stats



Do I have to choose a single grain for each database?

- Not at all; a database can have multiple tables with different grains, each one serving a different reporting need



What happens if I pick the wrong grain?

- Some queries may require additional joins or aggregations, hurting performance
- Some metrics may be impossible to calculate accurately, and you may need to rebuild the table

*Copyright Maven Analytics, LLC

PRIMARY KEYS

A **primary key** is a single column *(or multiple columns)* with values that uniquely identify each row

- Primary keys have two requirements – every value must be unique and no NULL values are allowed
- Every table must have a primary key

Primary key: ID column

ID	Date	Customer	Product	Quantity
101	5/1/26	C001	P19	1
102	5/1/26	C002	P17	1
103	5/1/26	C002	P18	NULL
104	5/1/26	C003	P16	5
105	5/2/26	C001	P19	1
106	5/2/26	C004	P17	3

Primary key*: Date + Region + Product columns

Date	Region	Product	Units	Revenue
5/1/26	North	P17	29	$43.50
5/1/26	South	P17	43	$64.50
5/2/26	North	P17	52	$78.00
5/2/26	South	P17	71	$106.50
5/3/26	North	P17	48	$72.00
5/3/26	South	P17	65	$97.50

This is called a **composite key*

*Copyright Maven Analytics, LLC

PRIMARY KEYS: NATURAL VS SURROGATE

There are **two types of primary keys**: Natural & Surrogate

- **Natural key:** The value comes from source data *(Customer_Email)*
- **Surrogate key:** The value is artificially generated *(Customer_ID)*

Surrogate	*Natural*		
Customer_ID	Customer_Email	First_Name	Last_Name
C001	ana@conda.com	Ana	Conda
C002	bobbyt@aol.com	Bobby	Tables
C003	ca@gmail.com	Cat	Alog
C004	datadiva@aol.com	Dana	Davis
C005	etl@yahoo.com	Emmett	Thomas-Lee
C006	function@gmail.com	Max	Function



The main problems of using natural keys are that they're not always truly unique, they can change over time, and they're often large

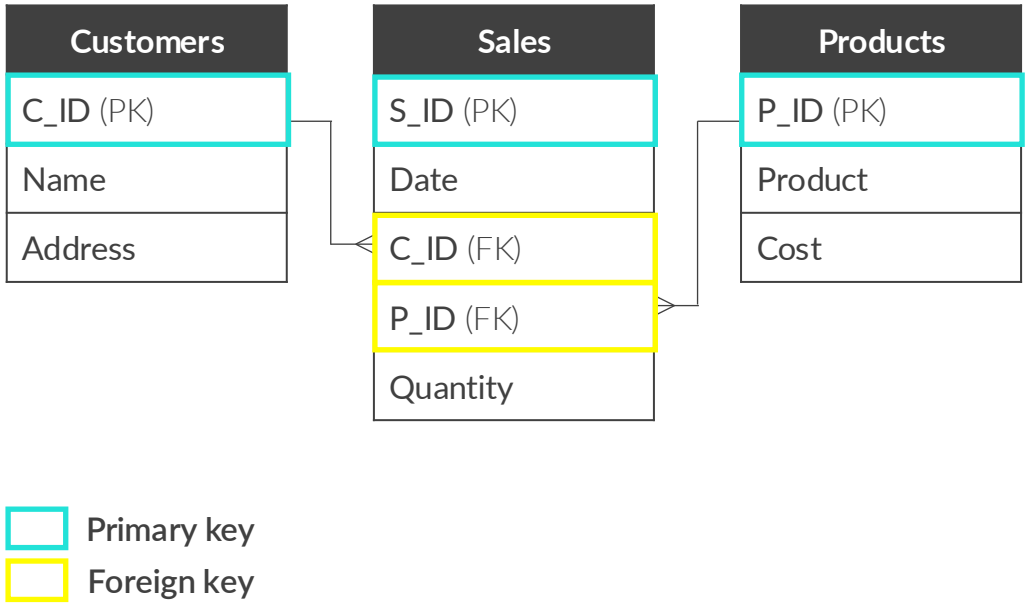
Surrogate keys solve all these problems, so it is **standard practice to add them** to tables in OLAP systems and data warehouses

*Copyright Maven Analytics, LLC

FOREIGN KEYS

A **foreign key** is a column that links to the primary key in another table

- In other words, foreign keys are how tables in a database connect to one another
- Unlike primary keys, not every table needs a foreign key



Products

P_ID	Product	Cost
P16	Pretzels	$3.99
P17	Chocolate	$4.50
P18	Chips	$2.99
P19	Soda	$1.50

Sales

S_ID	Date	C_ID	P_ID	Quantity
101	5/1/26	C001	P19	1
102	5/1/26	C002	P17	1
103	5/1/26	C002	P18	1
104	5/1/26	C003	P16	5
105	5/2/26	C001	P19	1
106	5/2/26	C004	P17	3

*Copyright Maven Analytics, LLC

TABLE DESIGN FAQs



Why is table design so important?

- Poor table design results in inaccurate metrics, broken joins, and performance issues that are time-consuming, costly, and hard to debug



What if you mess up or design a table incorrectly?

- Don't worry – (1) trial & error is part of the process, and (2) there's no single "correct design"
- If designing new tables, you can drop and recreate them easily
- If modifying existing tables, you'll likely need to update any pipelines or models built on top of them



What are good naming conventions?

- Tables should be plural and descriptive *(customers, sales, products, etc.)*
- Columns should be singular *(customer_id, product_name, etc.)*
- Table and column names generally use **snake_case**, but you may occasionally see **CamelCase** in older systems
- Avoid SQL reserved words as column names — words like "date" or "type" can cause unexpected errors

*Copyright Maven Analytics, LLC

FACT & DIMENSION TABLES

*Copyright Maven Analytics, LLC

FACT & DIMENSION TABLES

Fact & dimension tables are the core of analytical data

- In OL**T**P data models, tables capture individual **transactions**
- In OL**A**P data models, transactions are organized into fact and dimension tables for **analysis**

fct_sales

S_Key	S_ID	D_Key	C_Key	P_Key	Quantity	Revenue
SK1	101	DK260501	CK1	PK19	1	$1.50
SK2	102	DK260501	CK2	PK17	1	$4.50
SK3	103	DK260501	CK2	PK18	1	$2.99
SK4	104	DK260501	CK3	PK16	5	$19.95
SK5	105	DK260502	CK1	PK19	1	$1.50
SK6	106	DK260502	CK4	PK17	3	$13.50
SK7	107	DK260502	CK5	PK18	2	$5.98

*This **fact table** stores the central sales metrics at the desired grain*

dim_customer

C_Key	C_ID	Name	Address
CK1	C001	Ava	12 Main Street
CK2	C002	Ben	45 Oak Avenue
CK3	C003	Cal	7 Maple Road
CK4	C004	Deb	301 Pine Street
CK5	C005	Eve	88 Elm Drive

*This **dimension table** stores the descriptive attributes of each customer*

*Copyright Maven Analytics, LLC

FACT TABLES

Fact tables capture the measurable events that happen in your business

fct_sales

S_Key	S_ID	D_Key	C_Key	P_Key	Quantity	Revenue
SK1	101	DK260501	CK1	PK19	1	$1.50
SK2	102	DK260501	CK2	PK17	1	$4.50
SK3	103	DK260501	CK2	PK18	1	$2.99
SK4	104	DK260501	CK3	PK16	5	$19.95
SK5	105	DK260502	CK1	PK19	1	$1.50
SK6	106	DK260502	CK4	PK17	3	$13.50
SK7	107	DK260502	CK5	PK18	2	$5.98

*A **surrogate key** is added to uniquely identify each row*

This makes joins more efficient when querying, and protects against natural key changes

*The main characteristic of a fact table is that they hold **metrics**, or qualitative data (sales, clicks, orders, etc.)*

*This is numeric data that can be further **aggregated when querying***

*The **natural key** from the original source systems is kept in case you want to join back with the source system, or you want to use the field for reporting*

*These **foreign keys** link to dimension tables*

*Copyright Maven Analytics, LLC

SIDE NOTE: TRANSACTION TABLE VS. FACT TABLE

Example Website pageviews data

OLTP Transaction Table

click_id	user_id	session_id	timestamp	page_url	ad_id	ad_campaign	browser	device	referrer_url	ip_address
1	U001	S001	2026-05-01 09:01	/home	A101	SpringPromo	Chrome	Desktop	google.com	192.168.1.1
2	U001	S001	2026-05-01 09:02	/product/widget-a	A101	SpringPromo	Chrome	Desktop	/home	192.168.1.1
3	U001	S001	2026-05-01 09:05	/cart	NULL	NULL	Chrome	Desktop	/product/widget-a	192.168.1.1

OLAP Fact Table

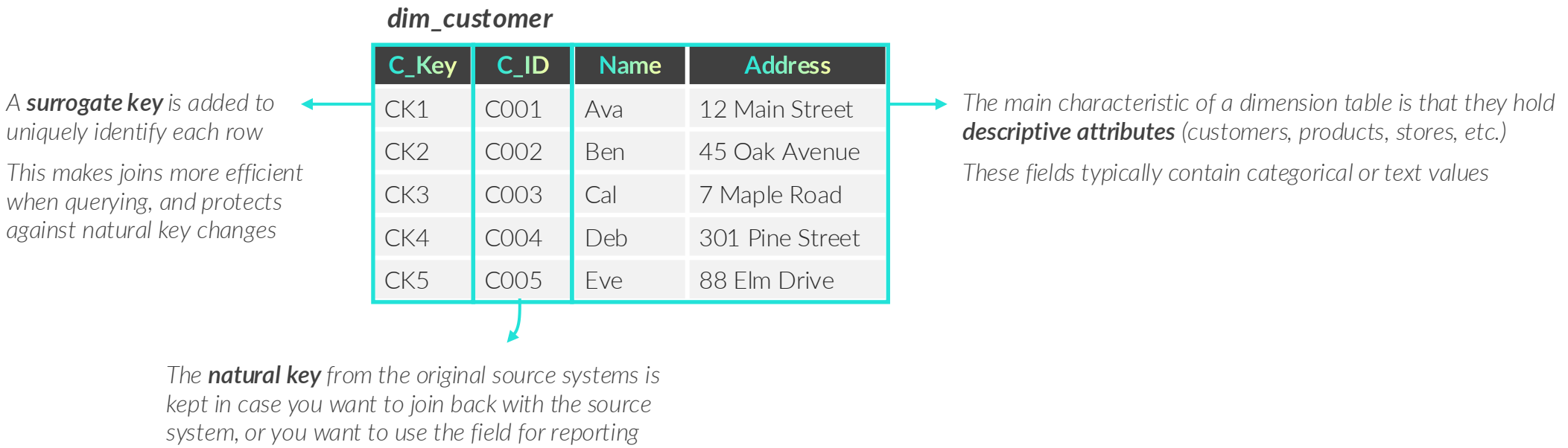
user_id	event_date	event_hour	page_url	session_duration	ad_views
U001	2026-05-01	9	/home	1 min	1
U001	2026-05-01	9	/product/widget-a	3 min	1
U001	2026-05-01	9	/cart	4 min	0

*Notice that the grain is the same in both cases, but the goal of the OLTP transaction table is to **log all the event details**, while the OLAP fact table is curated to be **easy for analysis***

*Copyright Maven Analytics, LLC

DIMENSION TABLES

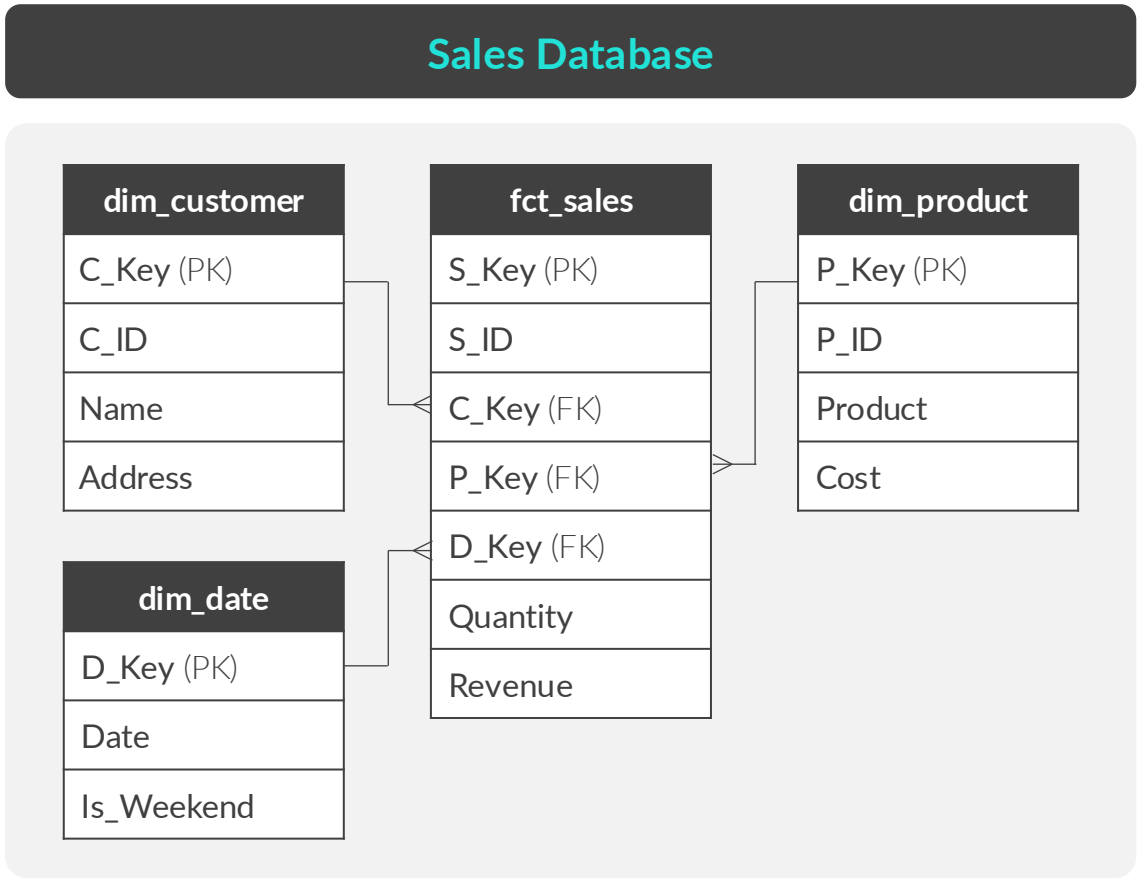
Dimension tables, also known as lookup tables, provide the context around each event



*Copyright Maven Analytics, LLC

FACT & DIMENSION TABLES TOGETHER

Data warehouses contain **fact and dimension tables** that are connected to one another



*This arrangement, with the fact table in the middle and dimension tables surrounding it, is called a **star schema***

*It's a common **schema design** found in analytical system (more coming up next!)*

Notice the naming convention for fact and dimension tables – **fct_plural** and **dim_singular**

*Copyright Maven Analytics, LLC

SCHEMA DESIGNS

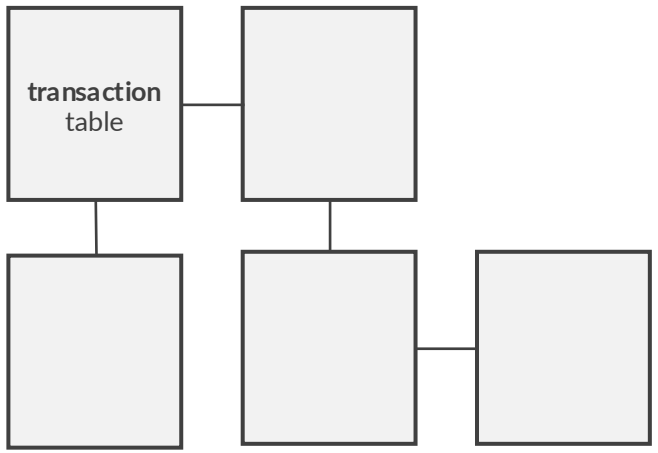
*Copyright Maven Analytics, LLC

SCHEMA DESIGNS

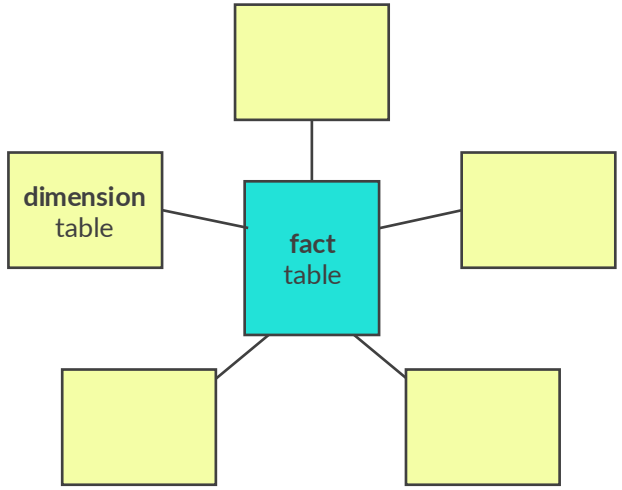
A **schema** is the blueprint or plan for how tables are organized in a database

- We'll cover three popular schema designs: Transactional, Star, and Snowflake schemas

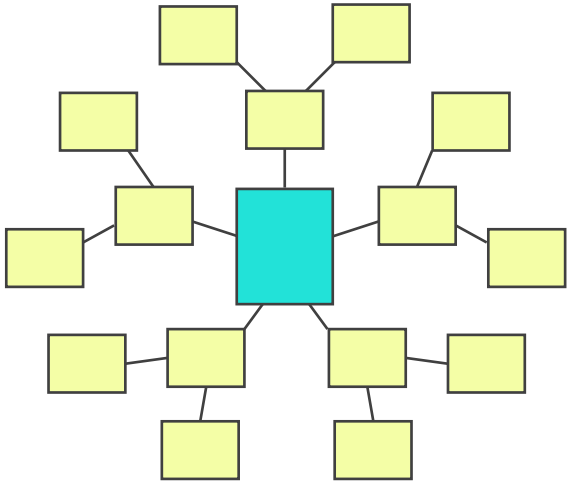
Transactional Schema



Star Schema



Snowflake Schema



*Copyright Maven Analytics, LLC

NORMALIZATION

Normalization is a core concept in schema design that describes the decision to either split data across multiple tables or keep it together in one

Normalized tables

*Often found in **OLTP** databases*

- Optimized for write performance and data integrity
- Eliminates redundancy
- Requires many joins to reconstruct information

C_ID	Name
C001	Ava
C002	Ben

P_ID	Product	Cost
P16	Pretzels	$3.99
P17	Chocolate	$4.50

*The most common level of normalization is **3NF (third normal form)**, where every piece of data is stored in exactly one place*

Denormalized tables

*Often found in **OLAP** databases*

- Optimized for read performance and analytical queries
- Duplicates data and pre-joins relationships for query speed
- Trades more storage (bad) for better performance (good)

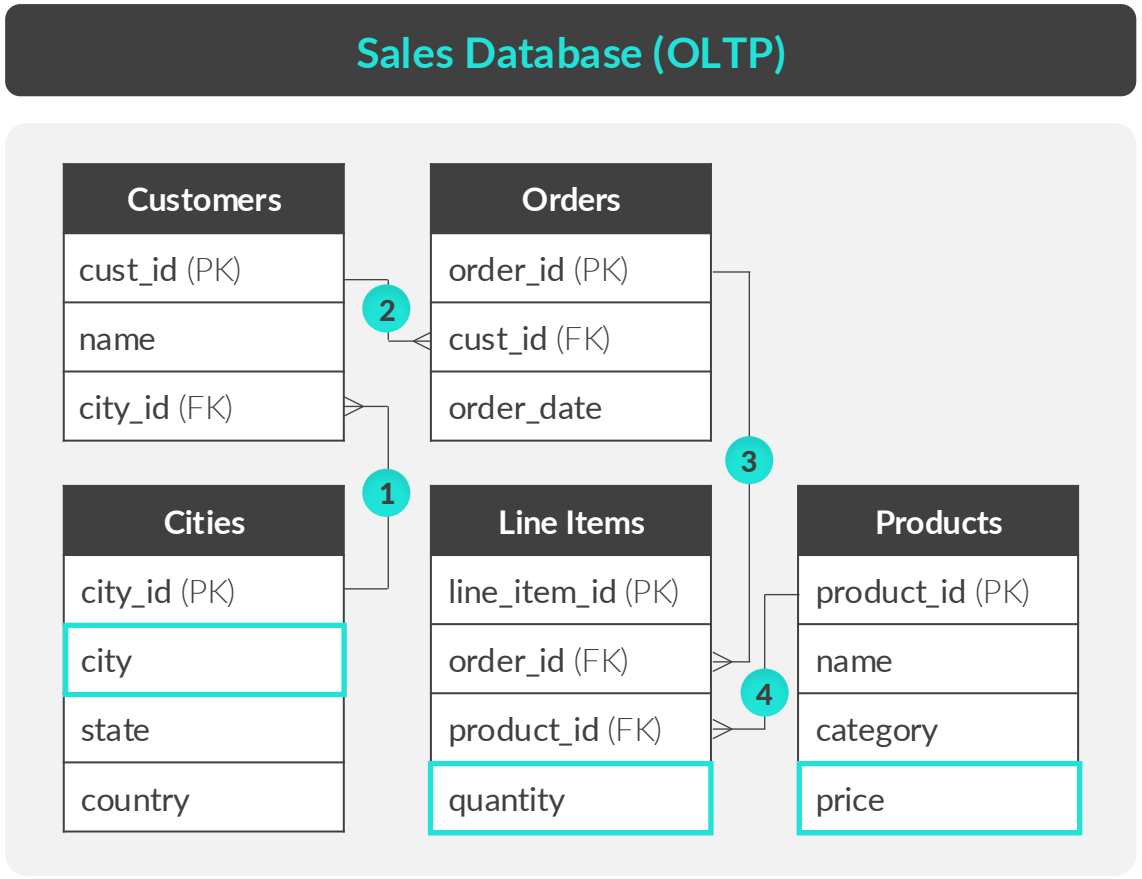
C_ID	Name	Date	Product	Amount
C001	Ava	2026-05-01	Pretzels	$3.99
C001	Ava	2026-05-08	Pretzels	$3.99
C002	Ben	2026-05-01	Chocolate	$4.50

*Notice the **duplicated data** – if something changes, you'd have to update multiple values instead of one*

*Copyright Maven Analytics, LLC

TRANSACTIONAL SCHEMA (OLTP)

Transactional schemas are designed for speed and accuracy, which means they are highly normalized to avoid data duplication and ensure every record is captured reliably



Data is stored as is as it comes into the OLTP database

- Each value is recorded once
- There are no aggregated measures
- This is optimized for inserts, updates and deletes

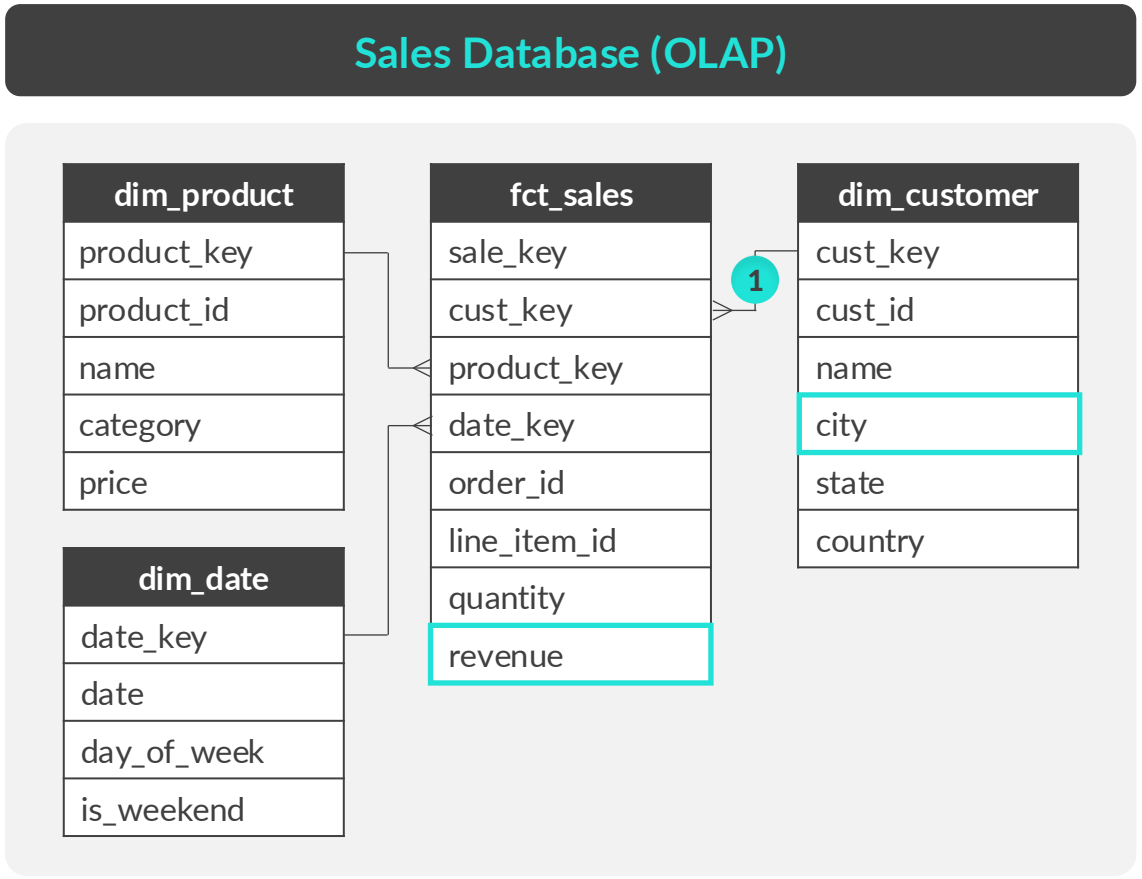
Multiple joins would be needed for querying this data, since it's spread across many normalized tables

*To get the total revenue by city, you'd have to start with cities to get the **city name**, then link to customers, then orders, then line items to get the **quantity** and products to get the **price** to calculate the **total revenue** (4 joins!)*

*Copyright Maven Analytics, LLC

STAR SCHEMA (OLAP)

Star schemas are designed for analysis, which means they have denormalized fact and dimension tables to simplify queries and make the data easy to explore



This schema takes extra storage, but makes queries easier to write and faster to execute *(optimized for analytics)*

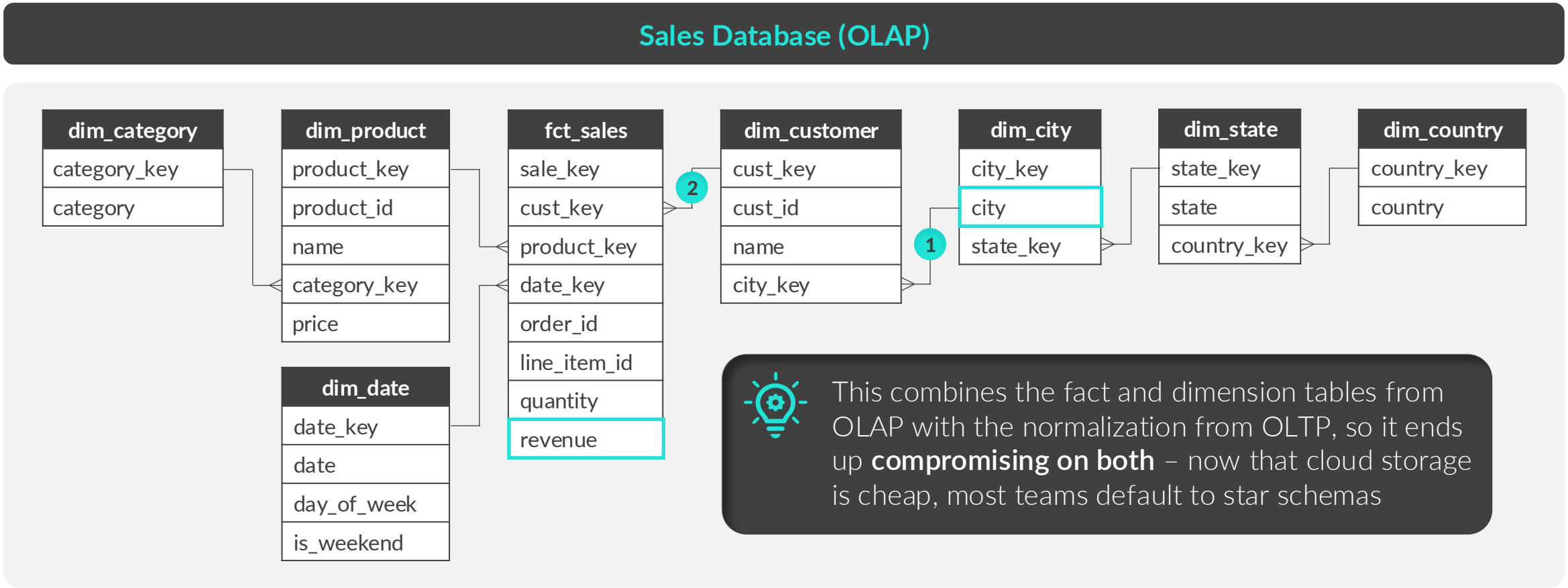
- The **fact table** stores measures and foreign keys
- The **dimension tables** store denormalized lookup values to reduce the number of joins when querying

This is the **most popular schema type found in data warehouses** – it's fast, simple & easy to query

*Copyright Maven Analytics, LLC

SNOWFLAKE SCHEMA (OLAP)

Snowflake schemas are a variation of a star schema with normalized dimension tables to reduce redundancy, at the cost of slightly more complex queries



*Copyright Maven Analytics, LLC

SCHEMA DESIGNS FAQs



Which schema design is most commonly used in data warehousing?

- Star schemas are the most popular in data warehouses since they are fast, simple, and easy to query



Can you have multiple fact tables in a database?

- Yes! Each fact table usually represents a different business process or event *(sales, inventory, returns, etc.)*
- They can share dimensions or have independent ones



Can you mix and match star and snowflake schemas in a database?

- Yes! Many warehouses use stars for simplicity and snowflakes when dimensions are large or hierarchical
- You can combine both in the same database

*Copyright Maven Analytics, LLC

HANDLING DATA CHANGES

*Copyright Maven Analytics, LLC

HANDLING DATA CHANGES

It's important to design fact & dimension tables that can **handle data changes** as they occur

- Changes are most commonly are found in dimension tables *(Slowly Changing Dimensions)*
- Fact tables should technically be immutable, so changes are typically made by adding adjustment rows

dim_product

P_Key	P_ID	Product	Cost
PK16	P16	Pretzels	$3.99
PK17	P17	Chocolate	$4.50
PK18	P18	Chips	$2.99
PK19	P19	Soda	$1.50

What if the cost for soda changes?

fct_sales

S_Key	S_ID	D_Key	C_Key	P_Key	Quantity	Revenue
SK1	101	DK260501	CK1	PK19	1	$1.50
SK2	102	DK260501	CK2	PK17	1	$4.50
SK3	103	DK260501	CK2	PK18	1	$2.99
SK4	104	DK260501	CK3	PK16	5	$19.95
SK5	105	DK260502	CK1	PK19	1	$1.50
SK6	106	DK260502	CK4	PK17	3	$13.50
SK7	107	DK260502	CK5	PK18	2	$5.98

What if this quantity should actually be 4?

*Copyright Maven Analytics, LLC

DIMENSION TABLE CHANGES

Slowly changing dimensions (SCD) are strategies for tracking how dimension attributes change over time, with two types being the most common *(out of 8 total)*

SCD Type 1

- Common approach that **overwrites values**
- Used when you don't need past information

C_Key	C_ID	Name	Address
CK1	C001	Ava	123 Main Street

C_Key	C_ID	Name	Address
CK1	C001	Ava	789 Pine Road

This is the simplest approach – people writing queries can't tell the difference

SCD Type 2

- Most common approach that **preserves full history**
- Used when you need to track every change

P_Key	P_ID	Product	Cost	Valid_From	Valid_To	Is_Current
PK19	P19	Soda	$1.50	2026-01-01	2026-05-01	FALSE
PK19	P19	Soda	$1.75	2026-05-01	NULL	TRUE

Remember the goal for OLAP systems is easy querying – with tables in this format, analysts can easily write a query to select the current price (Is_Current = TRUE), while also being able to look up historical prices

*Copyright Maven Analytics, LLC

FACT TABLE CHANGES

Fact tables represent events that happened, so their values should technically be **immutable**

- If they absolutely must be modified, the approach is to add new records that reflect the necessary adjustments
- As a best practice, you should not delete or overwrite existing records in fact tables

Date	C_ID	Product	Quantity	Transaction_Type
2026-05-01	C001	Pretzels	5	ORIGINAL
2026-05-01	C001	Pretzels	1	ADJUSTMENT
2026-05-01	C001	Pretzels	-2	RETURN

The quantity for this transaction would now calculate as 4

With tables in this format, analysts can easily write a query to aggregate rows or filter based on transaction type

*Copyright Maven Analytics, LLC

SIDE NOTE: DATA CHANGES IN SOURCE SYSTEMS

Data changes come from **source systems** (OLTP databases) – this is where the inserts, updates, and deletes actually happen – and there are two common ways they are captured

UpdatedAt field

Records when the update happened *(current state only)*

CID	Name	Address	UpdatedAt
C001	Ava	789 Pine Road	2026-05-01

Ava's previous address of 123 Main Street is overwritten and lost – the goal with OLTP is to make sure the current data is accurate, so this is a fine choice

CDC (change data capture) log

Records all the changes *(append only)*

EventTime	Operation	CID	Address
2026-01-01	INSERT	C001	123 Main Street
2026-05-01	UPDATE	C001	789 Pine Road

In addition to knowing the current data, historical data is captured here too



What's possible in your OLAP model depends entirely on what your source system captures, so understanding how changes are recorded at the source is an important step in your dimensional modeling process

*Copyright Maven Analytics, LLC

HANDLING DATA CHANGES FAQs



Is it okay to keep so much historical data?

- It depends – by keeping all historical data, queries can slow down over time
- However, you may want to keep historical data to be able to look at trends over time
- Some regulations also cap retention periods



When is it okay to delete data?

- You don't need to keep everything – delete the data when it no longer serves a business purpose or when legally required



At the end of the day, the question is whether you want to **preserve or delete history**

- By preserving history, you can query past data
- By deleting history, your queries are simpler and faster

Both approaches work; it's whatever is best for your situation

*Copyright Maven Analytics, LLC

MODERN DATA WORKFLOW

*Copyright Maven Analytics, LLC

MODERN DATA WORKFLOW

The **modern data workflow** outlines the core activities of a modern data team

- In this section, we'll be highlighting where each role (data engineer, analytics engineer, and data analyst) adds value

Scope	Ingest	Transform	Consume
Decide what data & metrics are needed	Extract and load raw data (EL) 	Clean & model data for analysis (T) 	Data is ready for querying & visualization 

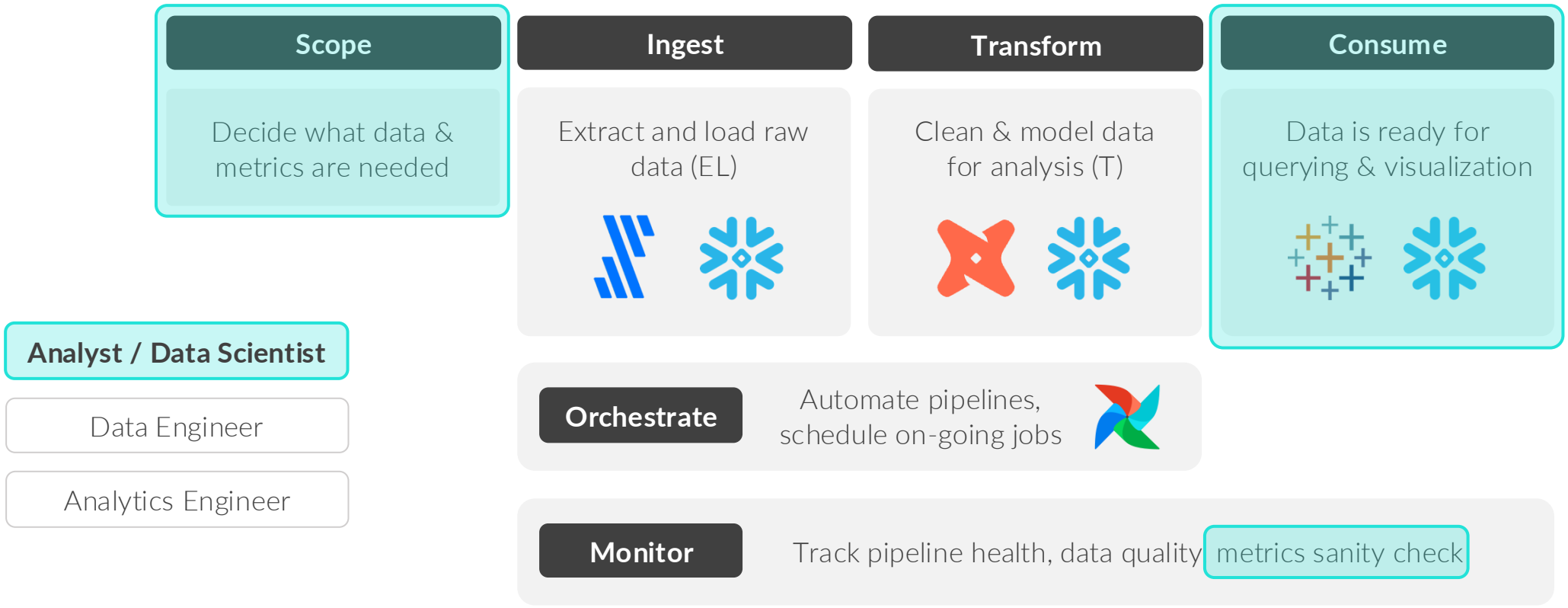
Orchestrate	Automate pipelines, schedule on-going jobs 
Monitor	Track pipeline health, data quality, metrics sanity check

*Copyright Maven Analytics, LLC

MODERN DATA WORKFLOW

The **modern data workflow** outlines the core activities of a modern data team

- In this section, we'll be highlighting where each role (data engineer, analytics engineer, and data analyst) adds value

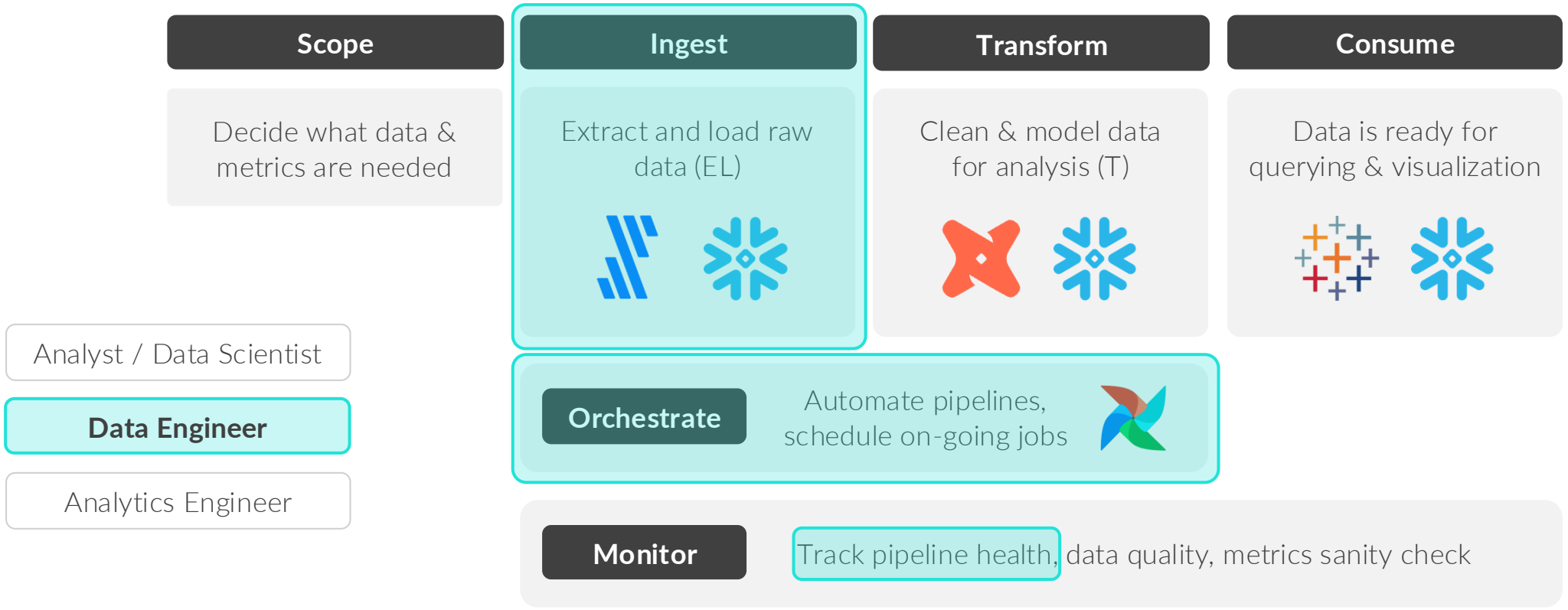


*Copyright Maven Analytics, LLC

MODERN DATA WORKFLOW

The **modern data workflow** outlines the core activities of a modern data team

- In this section, we'll be highlighting where each role (data engineer, analytics engineer, and data analyst) adds value

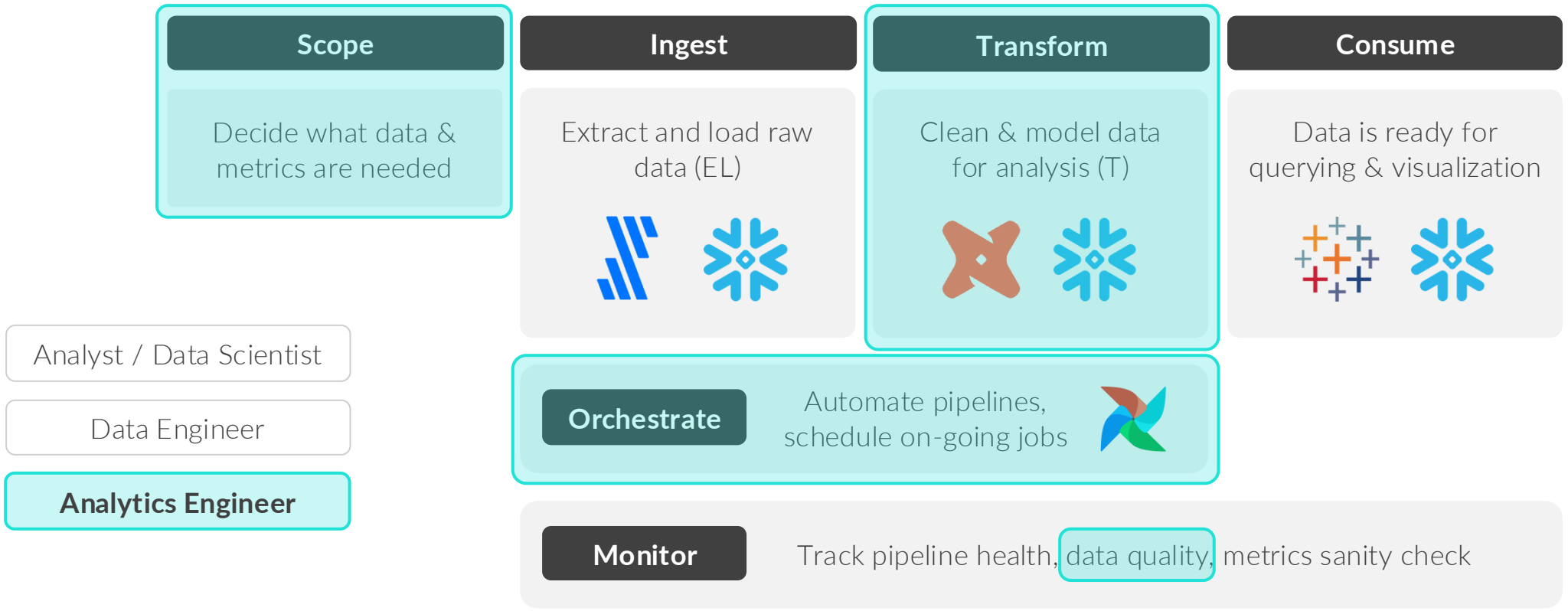


*Copyright Maven Analytics, LLC

MODERN DATA WORKFLOW

The **modern data workflow** outlines the core activities of a modern data team

- In this section, we'll be highlighting where each role (data engineer, analytics engineer, and data analyst) adds value



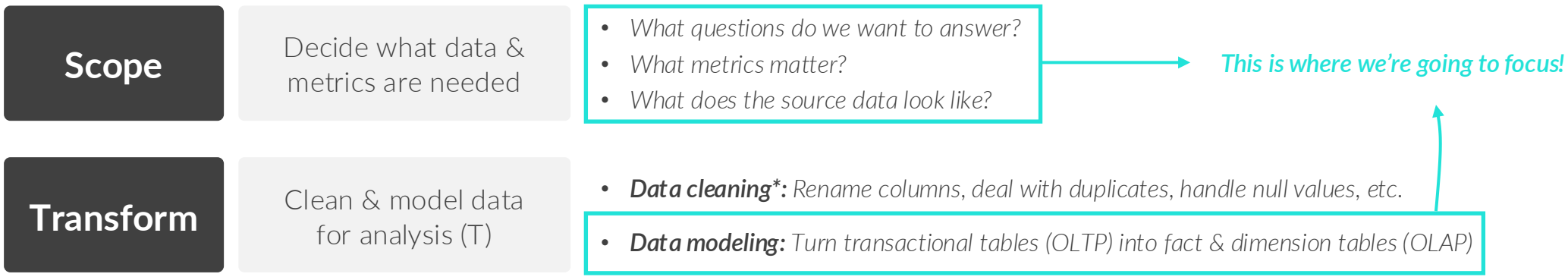
*Copyright Maven Analytics, LLC

DATA MODELING PROCESS

*Copyright Maven Analytics, LLC

DATA MODELING PROCESS

The **data modeling process** (*or dimensional modeling process*) represents the steps that an analytics engineer takes to design an OLAP dimensional model from an OLTP database



This is an iterative process – you likely won't get it right the first time!

*Data cleaning will be covered in the **Intro to dbt** course in this learning path*

*Copyright Maven Analytics, LLC

DATA MODELING PROCESS

The **data modeling process** (*or dimensional modeling process*) represents the steps that an analytics engineer takes to design an OLAP dimensional model from an OLTP database

1. Gather requirements and understand the source data
2. Identify fact tables and define their grain & metrics
3. Identify dimension tables and create a date table
4. Create surrogate keys and map them as foreign keys
5. Remove unnecessary columns and engineer new ones
6. Decide how to handle data changes *(SCD Type 1 or 2)*



This is a great place to **use GenAI to confirm if your table design looks good** and what tweaks you can incorporate to make it better

*Copyright Maven Analytics, LLC

PROJECT: DATA MODELING



THE **SITUATION**

You're an Analytics Engineer for **Maven Marketplace**, an online retailer

The sales team has an OLTP database that tracks transactions, but it's not optimized for reporting & analytics



THE **ASSIGNMENT**

Your task is to transform the OLTP database by **designing a proof of concept OLAP model** that shows how the data should be structured for analysis



THE **OBJECTIVES**

- Apply each step in the data modeling process *(from scoping to transformations)*
- Make key data modeling design decisions *(grain, keys, change handling, etc.)*
- Sketch a dimensional model that answers the business questions efficiently



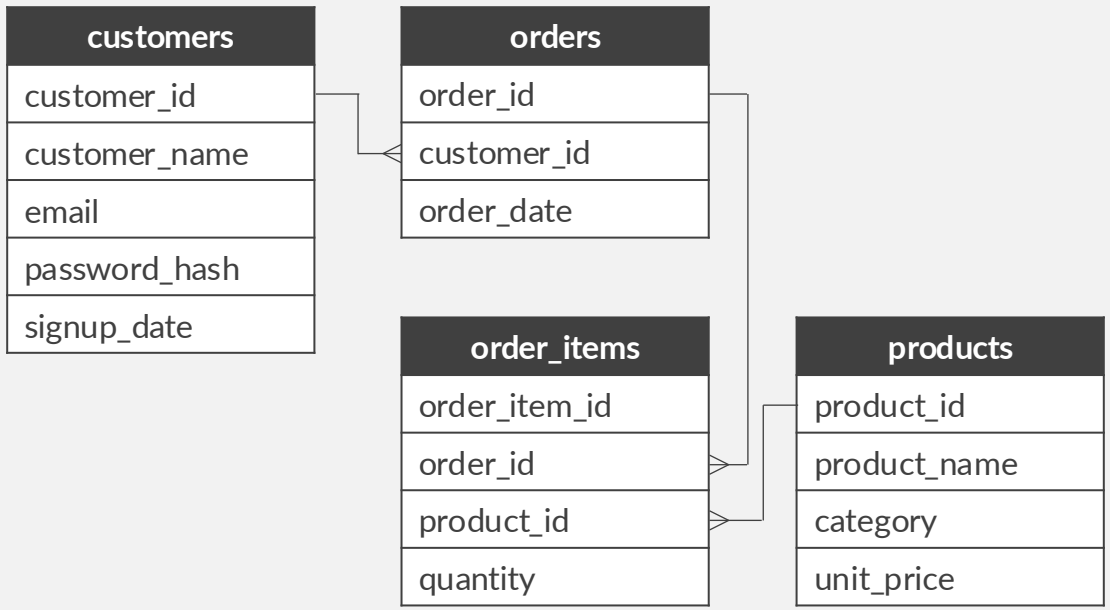
We'll be **designing these tables conceptually** for now, but we'll be building them using dbt and Snowflake later in the Analytics Engineering learning path

*Copyright Maven Analytics, LLC

PROJECT: DATA MODELING

1 Gather requirements and understand the source data

Maven Marketplace Database



What questions do we want to answer?

- Which customers are our top buyers?
- What are our most and least popular products in terms of revenue?
- How do sales trend over time?

What metrics matter?

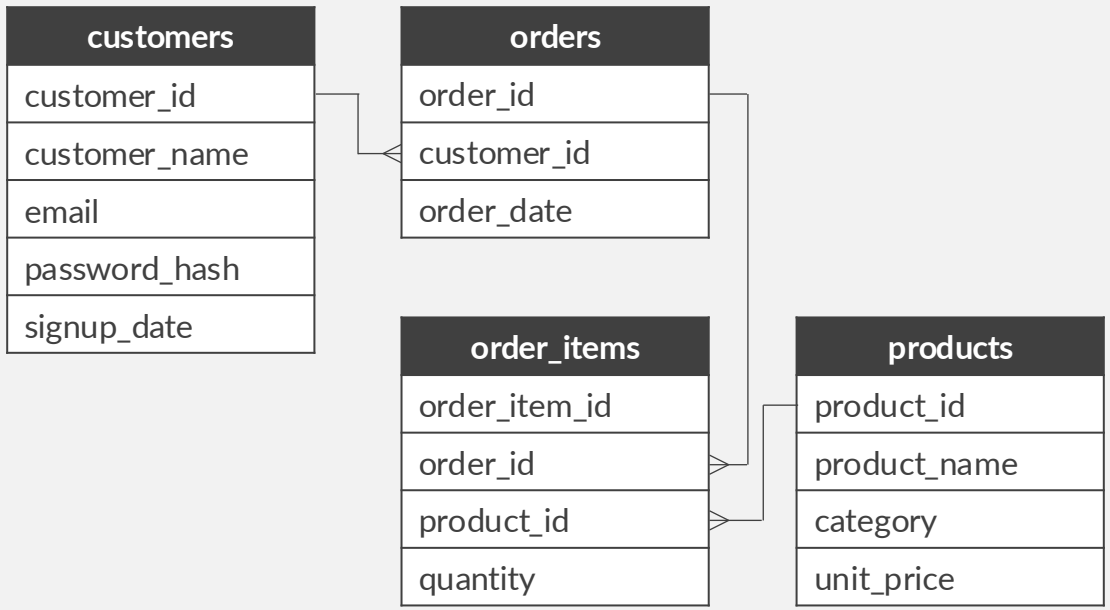
- Sales
- Revenue

*Copyright Maven Analytics, LLC

PROJECT: DATA MODELING

1 Gather requirements and understand the source data

Maven Marketplace Database



What questions de we want to answer?

- Which ***customers*** are our top buyers?
- What are our most and least popular ***products*** in terms of ***revenue***?
- How do ***sales*** trend over ***time***?

What metrics matter?

- *Sales*
- *Revenue*

*Copyright Maven Analytics, LLC

PROJECT: DATA MODELING

1 Gather requirements and understand the source data

Maven Marketplace Database

customers

customer_id	customer_name	email	password_hash	signup_date
501	Abby Johnson	abby.johnson@email.com	8f3kP9mNqL	2025-09-14
502	Bob Smith	bob.smith@email.com	xK2vR7tYue	2025-04-21
503	Carol Davis	carol.davis@email.com	HzD4mN8pLq	2025-03-30

orders

order_id	customer_id	order_date
1001	538	2025-02-25
1002	503	2026-01-26
1003	535	2026-03-26

order_items

order_item_id	order_id	product_id	quantity
1	1001	214	2
2	1002	217	1
3	1002	212	1

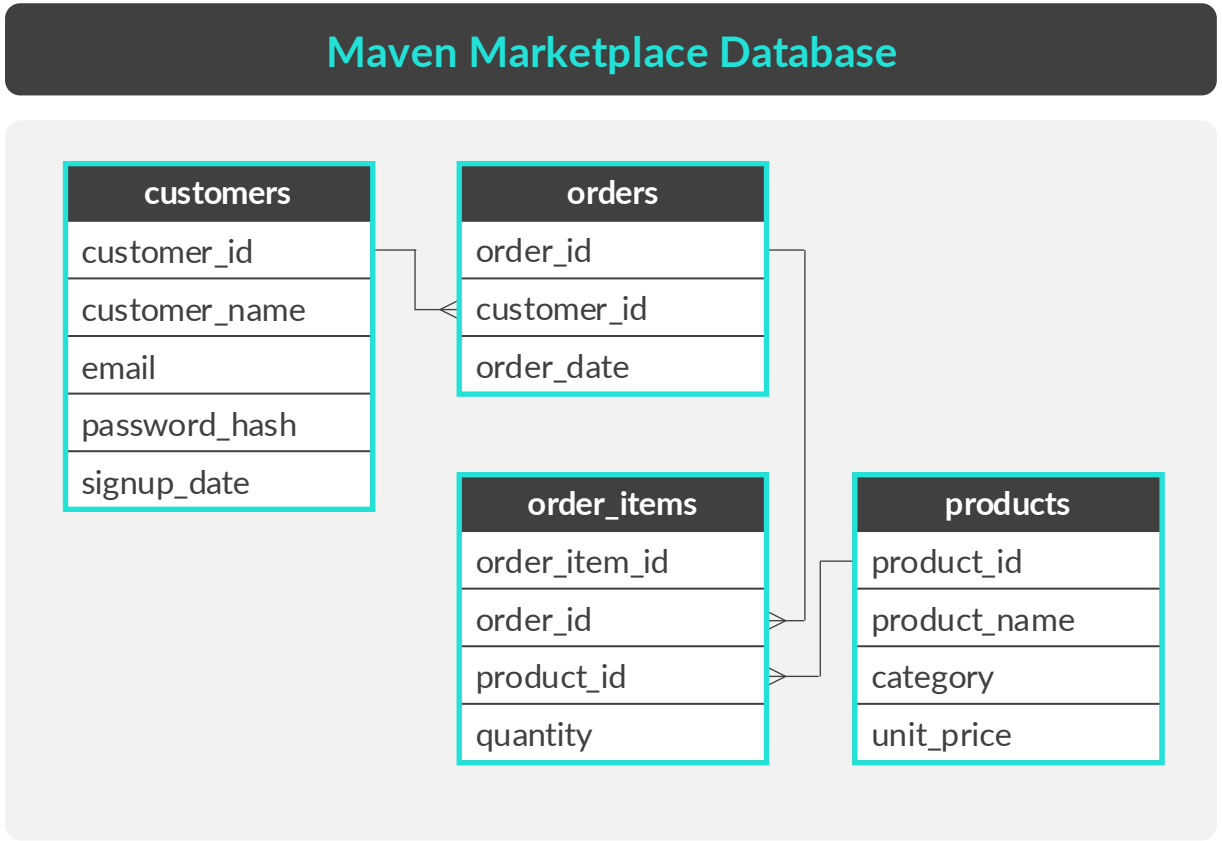
products

product_id	product_name	category	unit_price
201	Wireless Mouse	Electronics	$29.99
202	USB Keyboard	Electronics	$49.99
203	Notebook Set	Office Supplies	$15.99

*Copyright Maven Analytics, LLC

PROJECT: DATA MODELING

1 Gather requirements and understand the source data



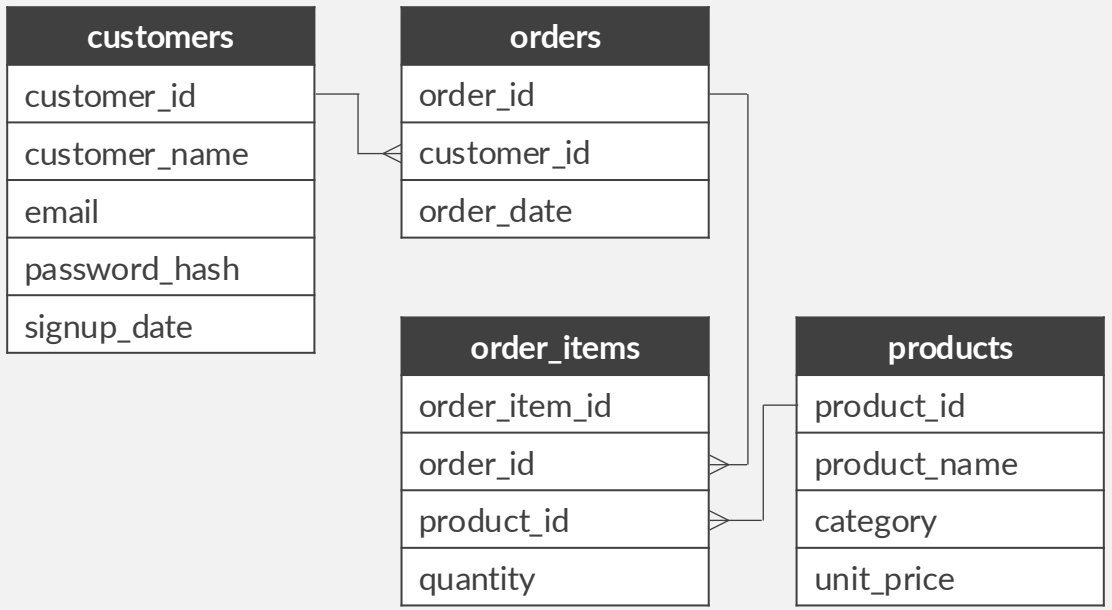
- The **orders** table registers individual customer orders
- The **order_items** table registers the product(s) in each order, where one order can have multiple order items
- The **customers** and **products** tables contain additional details on each

*Copyright Maven Analytics, LLC

PROJECT: DATA MODELING

2 Identify fact tables and define their grain & metrics

Maven Marketplace Database

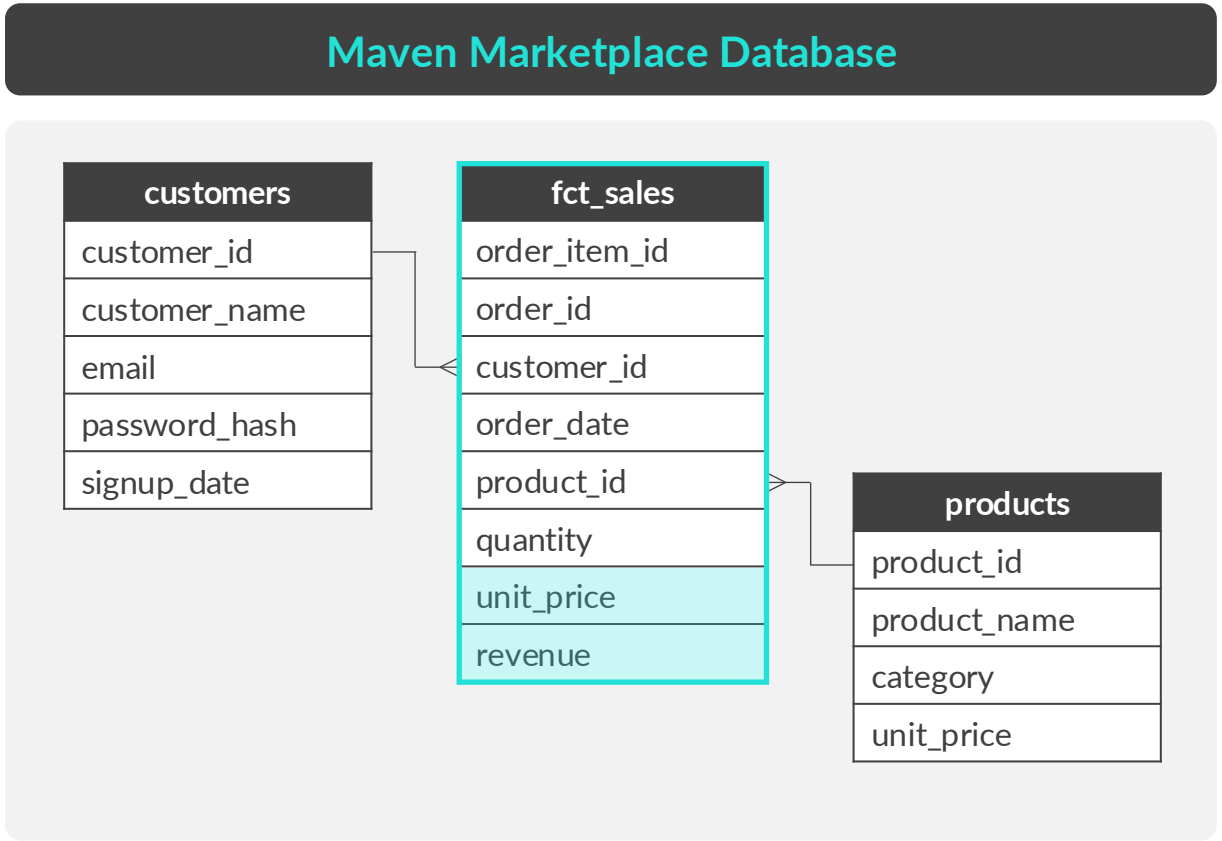


- Fact table: Combine the **orders** and **order_items** tables into a single fact table
- Grain: Keep from the **order_items** table, where each row represents a product purchased as part of an order
- Metrics: We need **sales** (quantity in the order_items table) and **revenue** (quantity in the order_items table x unit_price in the products table)

*Copyright Maven Analytics, LLC

PROJECT: DATA MODELING

2 Identify fact tables and define their grain & metrics

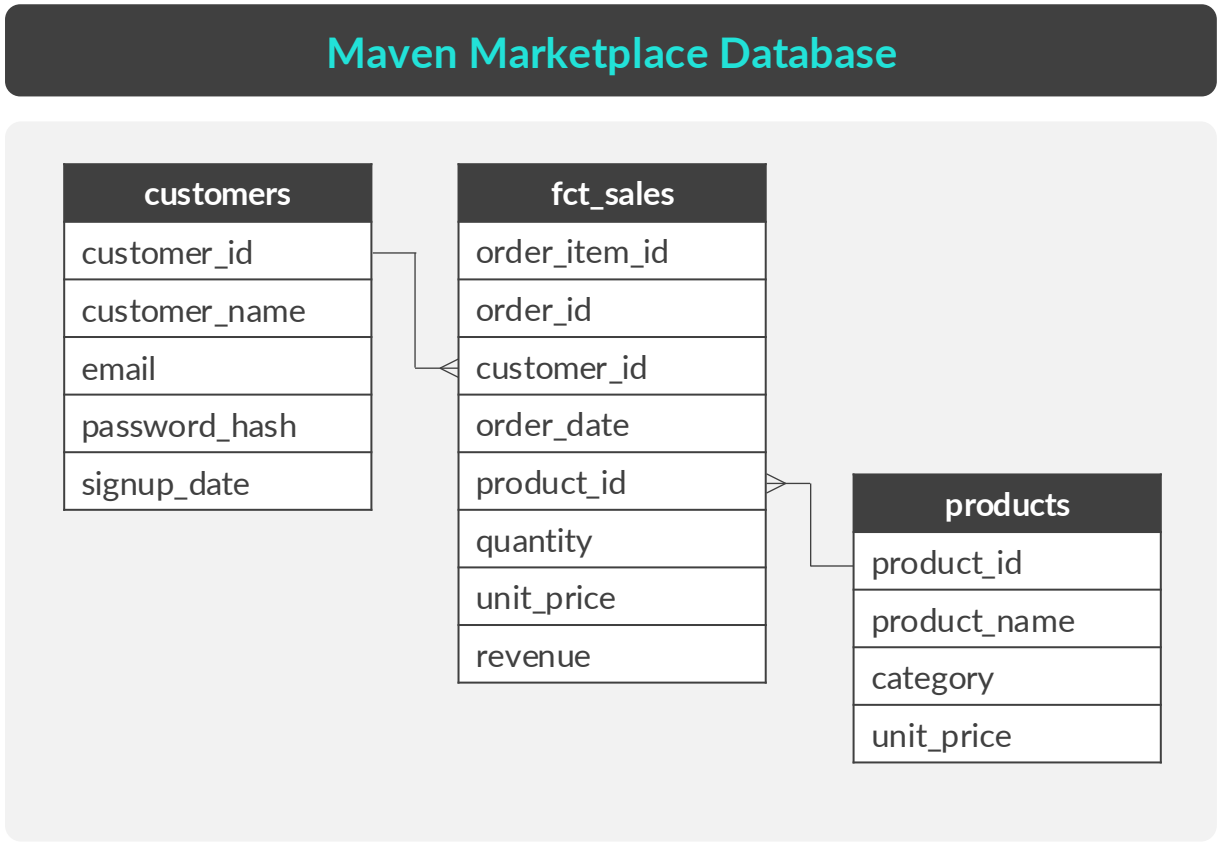


- Fact table: Combine the **orders** and **order_items** tables into a single fact table
- Grain: Keep from the **order_items** table, where each row represents a product purchased as part of an order
- Metrics: We need **sales** (quantity in the order_items table) and **revenue** (quantity in the order_items table x unit_price in the products table)

*Copyright Maven Analytics, LLC

PROJECT: DATA MODELING

3 Identify dimension tables and create a date table

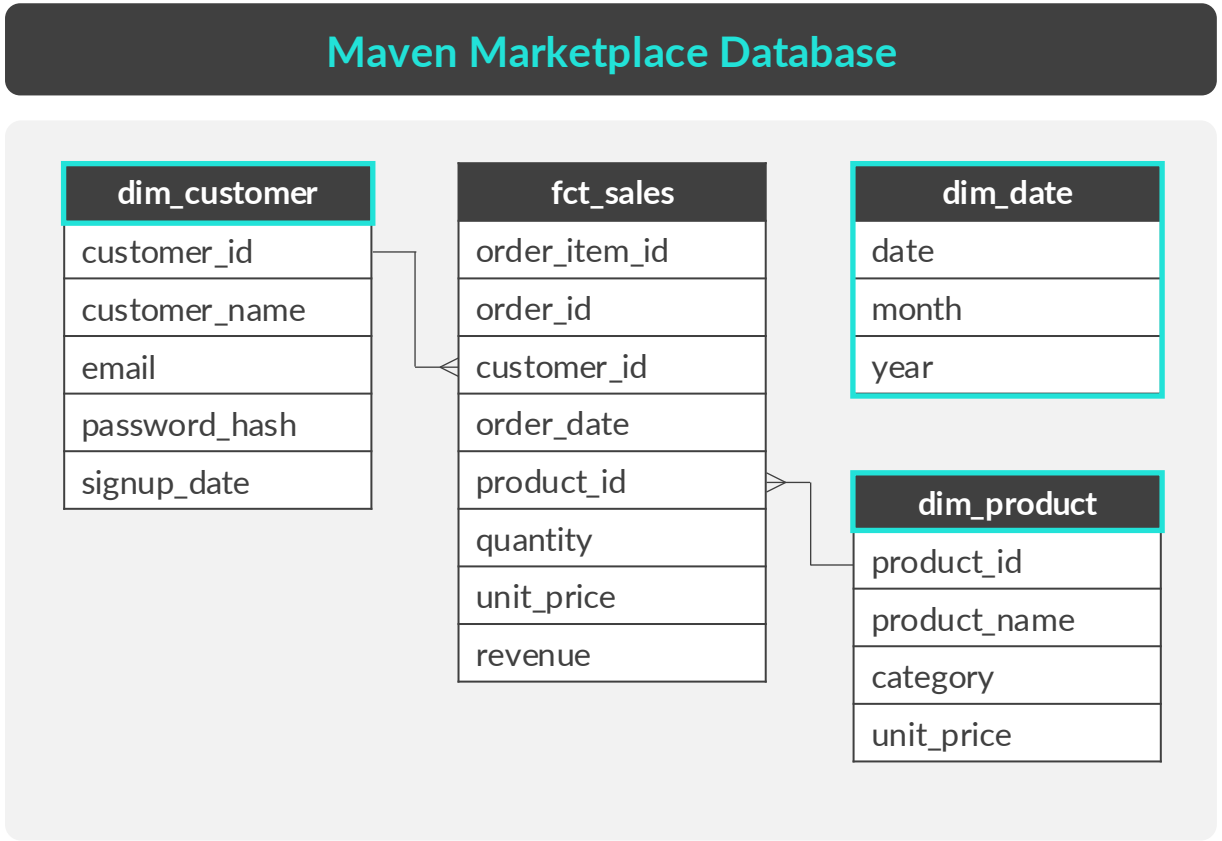


- Dimension tables: Change the names of the **customers** and **products** tables to start with dim_
- Date table: Create a new **dim_date** table, and add relevant attributes based on the requirements

*Copyright Maven Analytics, LLC

PROJECT: DATA MODELING

3 Identify dimension tables and create a date table

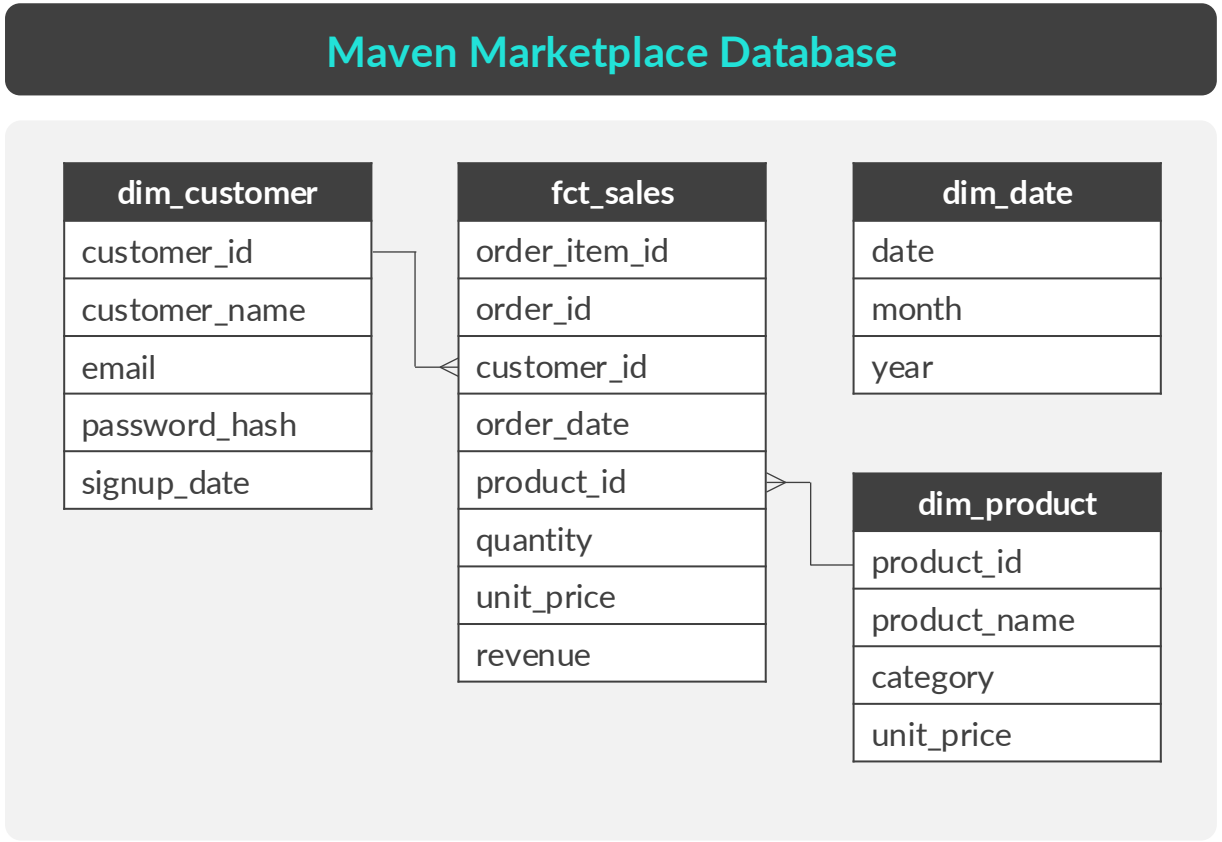


- Dimension tables: Change the names of the **customers** and **products** tables to start with dim_
- Date table: Create a new **dim_date** table, and add relevant attributes based on the requirements

*Copyright Maven Analytics, LLC

PROJECT: DATA MODELING

4 Create surrogate keys and map them as foreign keys



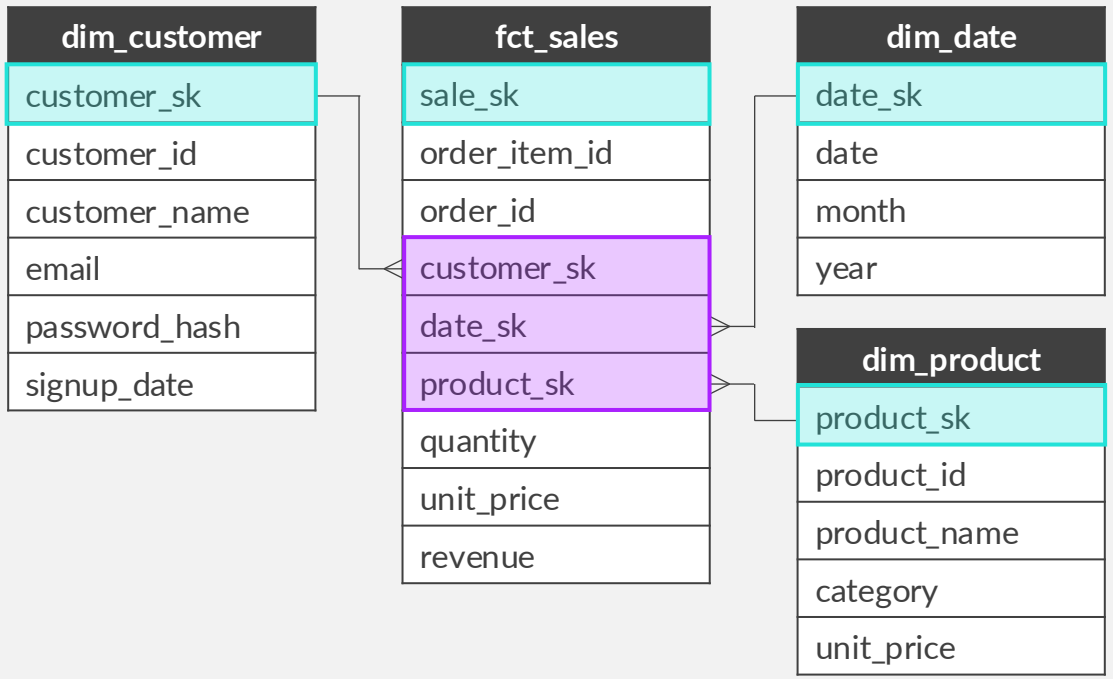
1. Add surrogate keys to the fact and dimension tables
2. Update the foreign keys in the fact table from the original IDs to their surrogate keys

*Copyright Maven Analytics, LLC

PROJECT: DATA MODELING

4 Create surrogate keys and map them as foreign keys

Maven Marketplace Database



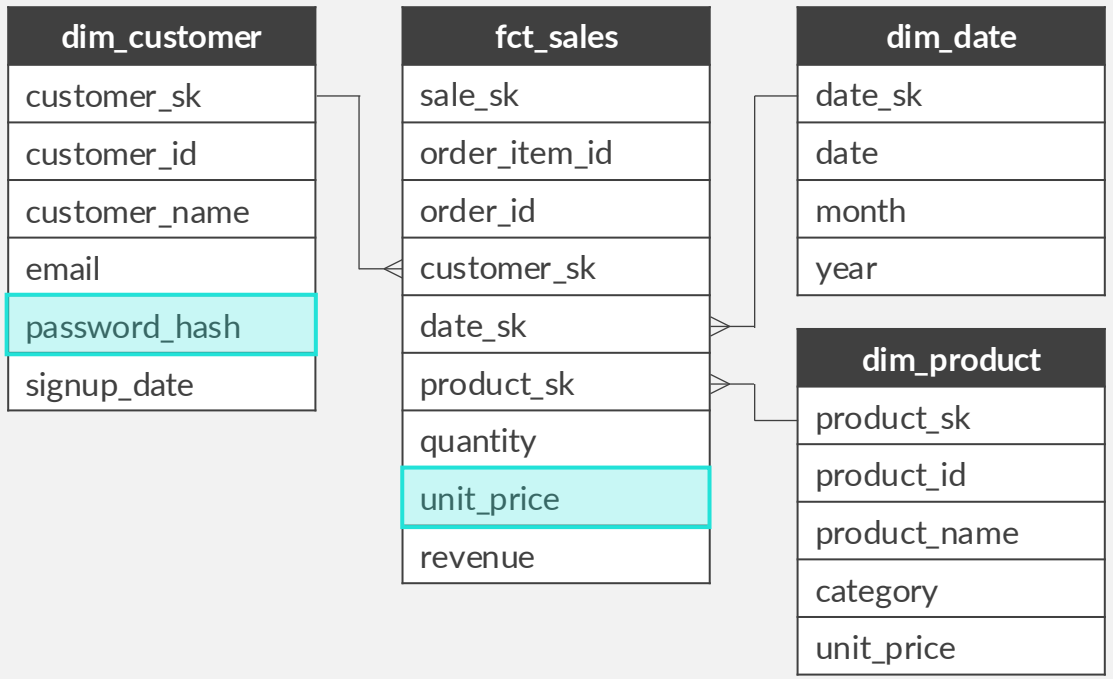
1. Add surrogate keys to the fact and dimension tables
2. Update the foreign keys in the fact table from the original IDs to their surrogate keys

*Copyright Maven Analytics, LLC

PROJECT: DATA MODELING

5 Remove unnecessary columns and engineer new ones

Maven Marketplace Database

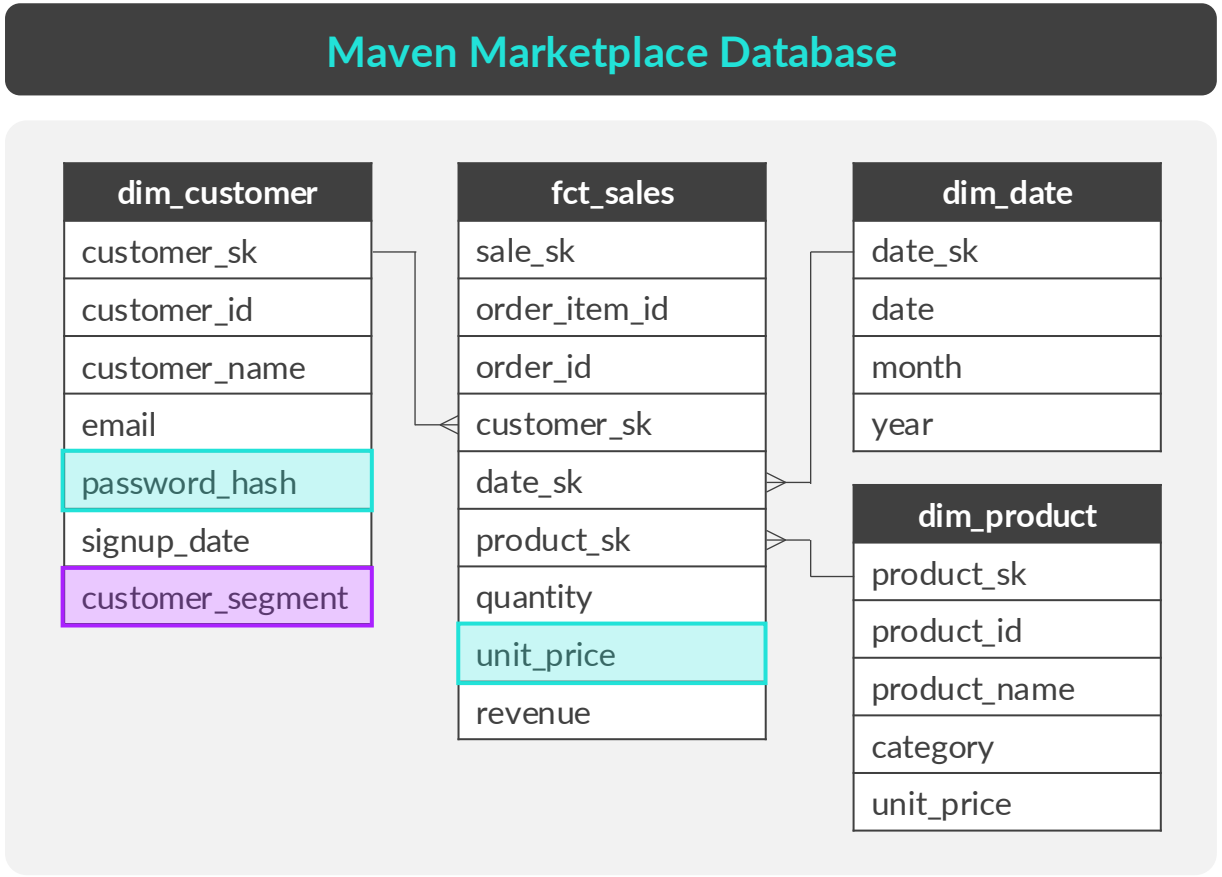


- The **password_hash** column adds no analytical value
- We used **unit_price** to calculate the **revenue**, but we no longer need it in the fact table

*Copyright Maven Analytics, LLC

PROJECT: DATA MODELING

5 Remove unnecessary columns and engineer new ones

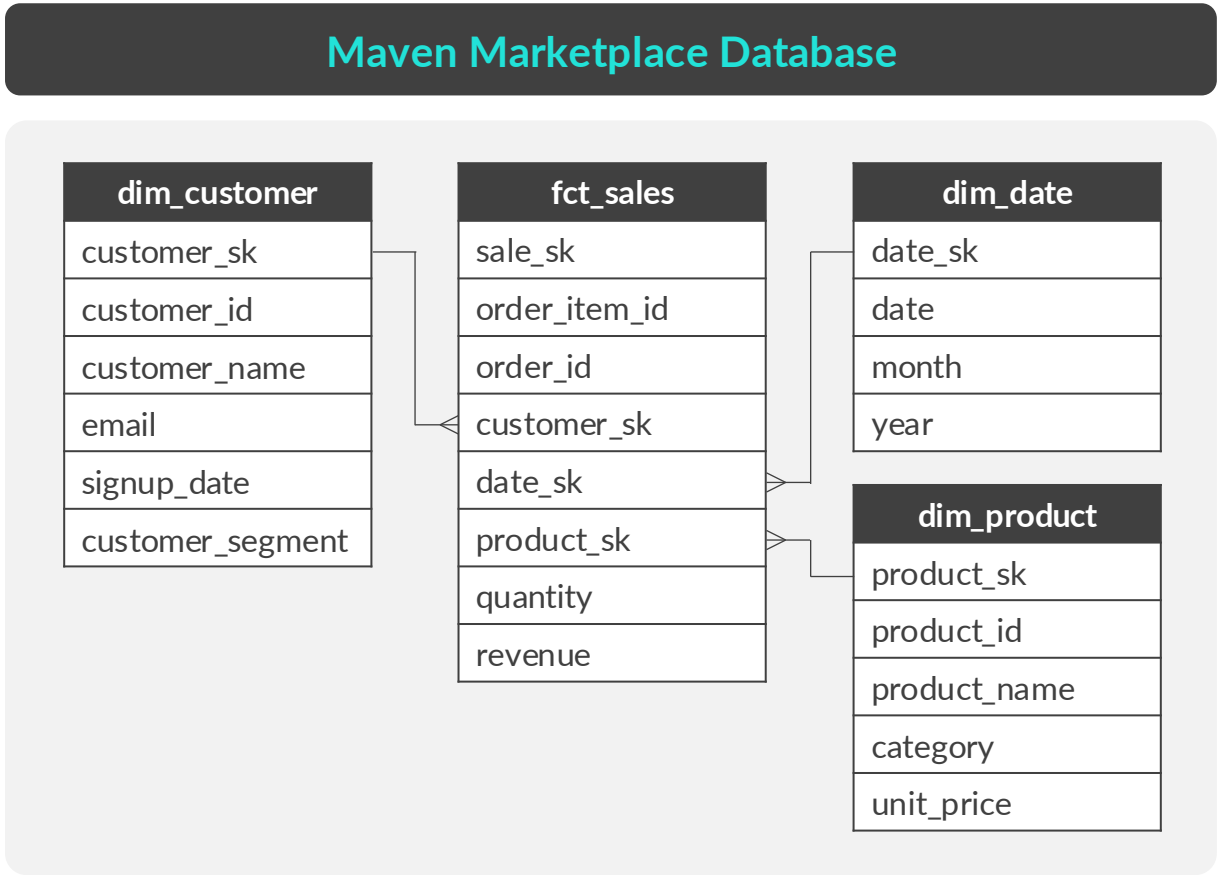


- The **password_hash** column adds no analytical value
- We used **unit_price** to calculate the **revenue**, but we no longer need it in the fact table
- We can engineer a new field called **customer_segment** with the logic:
  - *Inactive*: no orders in the past 180 days
  - *Premium*: total spend > $1,000 or > 10 orders
  - *Standard*: everyone else

*Copyright Maven Analytics, LLC

PROJECT: DATA MODELING

5 Remove unnecessary columns and engineer new ones

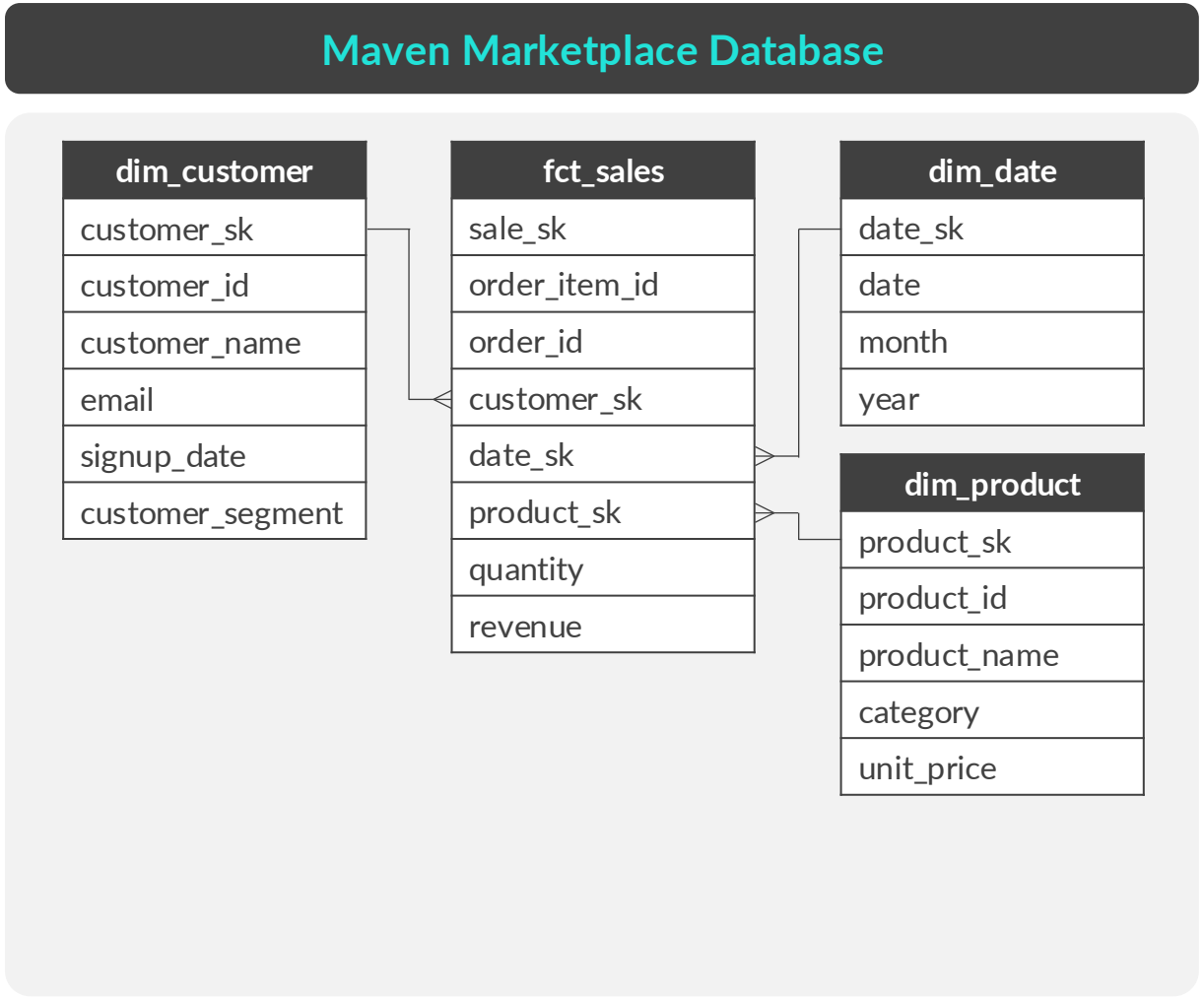


- The **password_hash** column adds no analytical value
- We used **unit_price** to calculate the **revenue**, but we no longer need it in the fact table
- We can engineer a new field called **customer_segment** with the logic:
  - *Inactive*: no orders in the past 180 days
  - *Premium*: total spend > $1,000 or > 10 orders
  - *Standard*: everyone else

*Copyright Maven Analytics, LLC

PROJECT: DATA MODELING

6 Decide how to handle data changes *(SCD Type 1 or 2)*

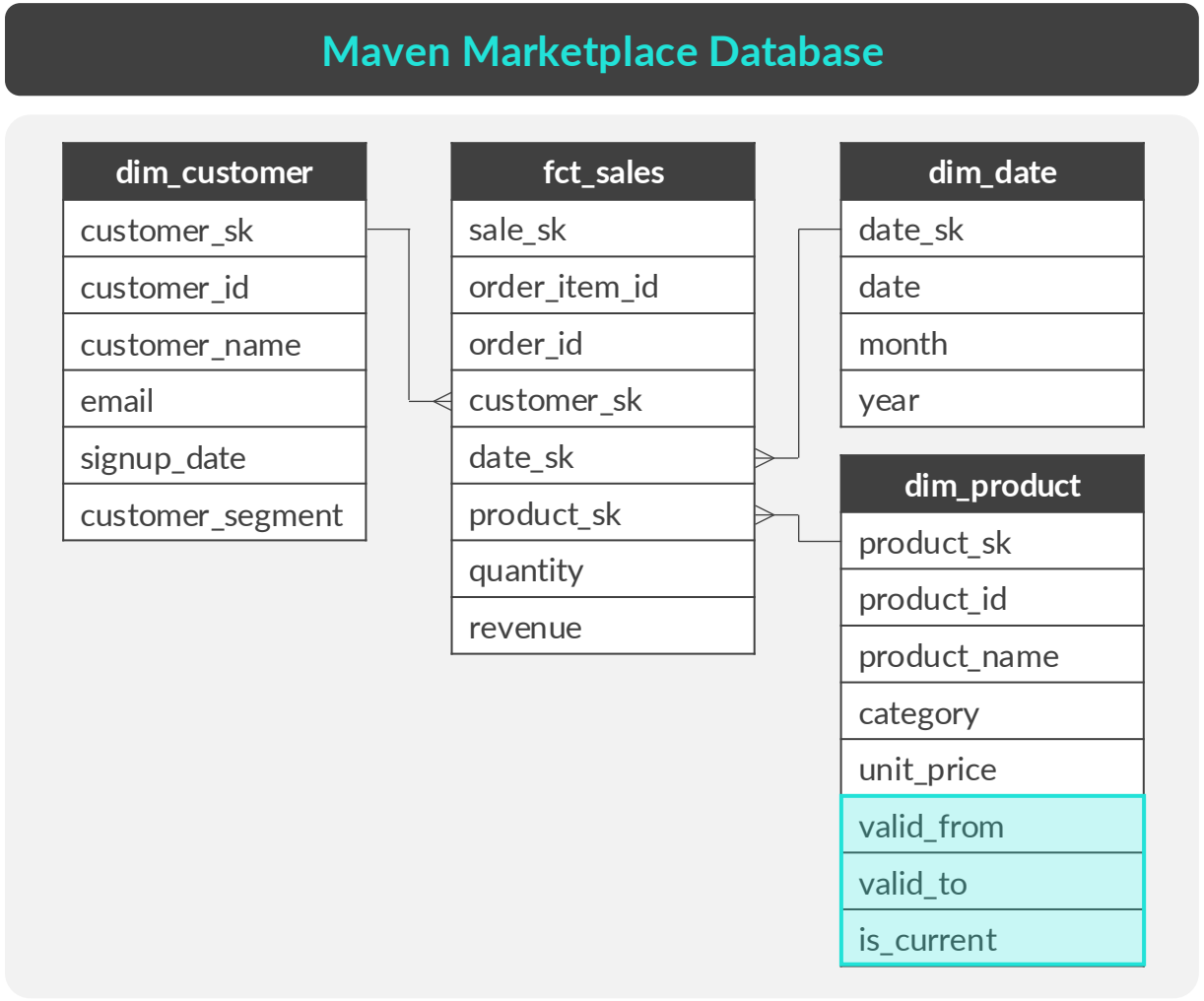


- Looking back at the OLTP source data, there is no CDC log, so it looks like changes overwrote historical values
- For most dimensions, we'll go with SCD Type 1 and just use the latest values

*Copyright Maven Analytics, LLC

PROJECT: DATA MODELING

6 Decide how to handle data changes *(SCD Type 1 or 2)*

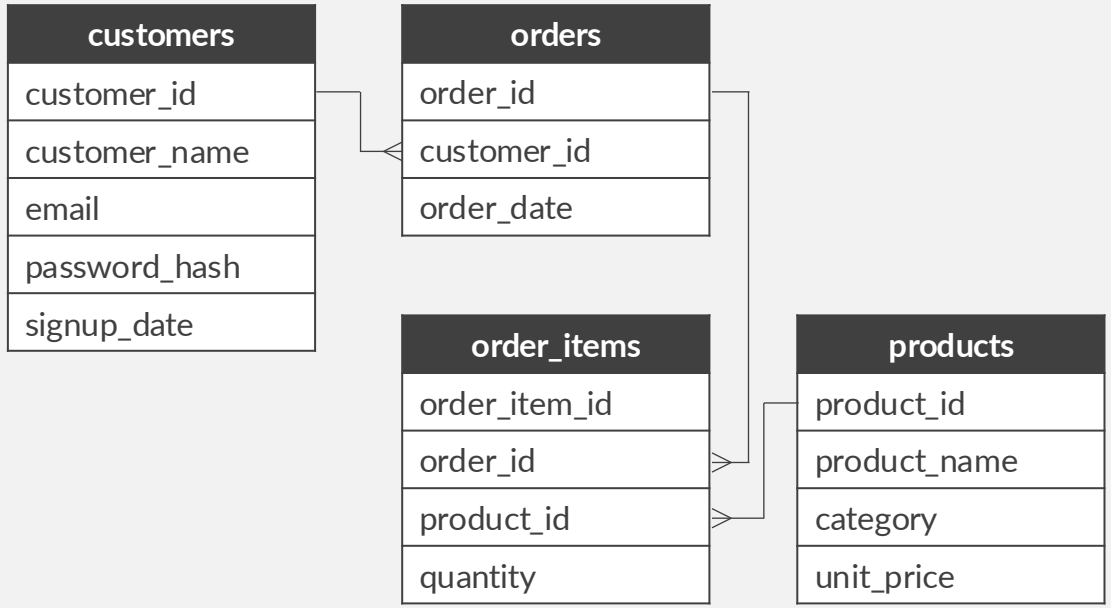


- Looking back at the OLTP source data, there is no CDC log, so it looks like changes overwrote historical values
- For most dimensions, we'll go with SCD Type 1 and just use the latest values
- But for product pricing, we'll implement SCD Type 2 — tracking effective start dates, end dates, and a current row flag so we can preserve price history over time

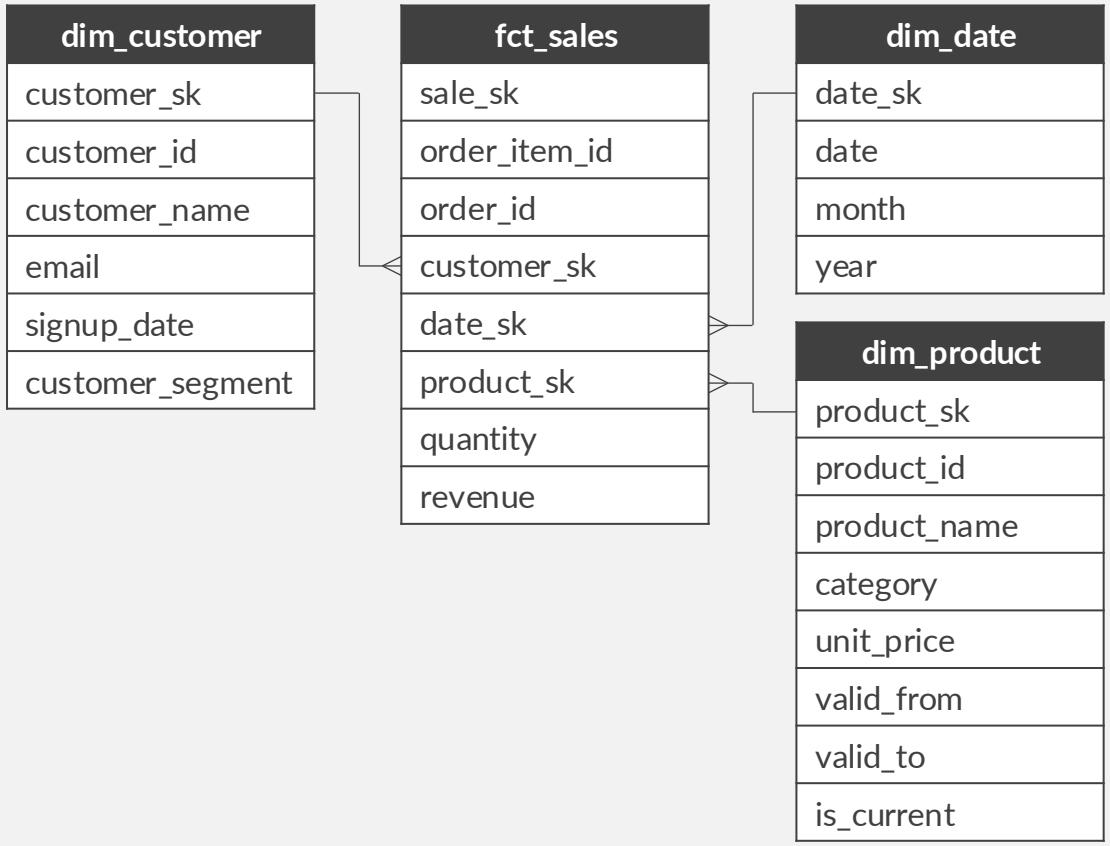
*Copyright Maven Analytics, LLC

PROJECT: DATA MODELING

Maven Marketplace Database (OLTP)



Maven Marketplace Database (OLAP)



*Copyright Maven Analytics, LLC

PROJECT: DATA MODELING

Congrats on designing your first dimensional model! 🎉

1 Use GenAI to review your design

- Once you've created the first draft of your model, this is a great time to use GenAI to confirm if your table design looks good and what tweaks you can incorporate to make your design better

2 There is no single "right" solution

- This solution is just one approach – yours may look different, and that's ok!
- In the real world, teams often design their OLAP tables differently
- What matters is capturing the right measures and dimensions for analysis

3 Want to actually code this up?

- Stick around for the rest of our Analytics Engineering learning path

*Copyright Maven Analytics, LLC

KEY TAKEAWAYS

*Copyright Maven Analytics, LLC

KEY TAKEAWAYS: DATA MODELING THEORY

A **data model** shows how data is organized and connected

- *Data models consist of tables and relationships, and the schema (or blueprint) will look different depending on whether the data is in an OLTP (transactional) system or an OLAP (analytical) system*

Designing tables for data warehouses is called **dimensional modeling**

- *Things to consider when creating tables include grain, primary keys, foreign keys, constraints, and more*

In OLAP systems, schemas are made up of **fact & dimension tables**

- *The star schema is the most popular design, with a fact table connected to dimension (or lookup) tables*
- *The snowflake schema is another design with additional, normalized dimensions to reduce redundancy*

Fact & dimension tables need to be designed to **handle data changes**

- *Dimension tables are expected to change (new address, price change, etc.) and SCD strategies can be used*
- *Fact tables should not change, but changes can happen by adding a new row with adjustments to the table*

*Copyright Maven Analytics, LLC

KEY TAKEAWAYS: DATA MODELING PRACTICE



The **modern data workflow** highlights the tasks of different data roles

- *Analysts scope & consume, data engineers ingest & orchestrate, and analytics engineers scope & transform*
- *All roles are required to monitor their tools to make sure the data pipeline is accurately moving data*



Analytics engineers must go through the **data modeling process** to turn raw source data into analytics-ready data for a data warehouse

- *Once a project is scoped, the focus moves to cleaning & transformation, including the data modeling steps*
- *These include gathering requirements, identifying the fact and dimension tables, defining the grain and metrics, mapping surrogate keys, removing and engineering columns, and deciding between SCD type 1 & 2*



You've now successfully **designed an OLAP data model** like an AE would

- *Next, let's actually implement the design using MDS tools – Snowflake, Fivetran, dbt, Airflow and Tableau*

*Copyright Maven Analytics, LLC